



DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving

Yinmin Zhong¹ Shengyu Liu¹ Junda Chen³ Jianbo Hu¹ Yibo Zhu² Xuanzhe Liu¹
Xin Jin¹ Hao Zhang³

¹School of Computer Science, Peking University ²StepFun ³UC San Diego

Abstract

DistServe improves the performance of large language models (LLMs) serving by disaggregating the prefill and decoding computation. Existing LLM serving systems colocate the two phases and batch the computation of prefill and decoding across all users and requests. We find that this strategy not only leads to strong prefill-decoding interferences but also couples the resource allocation and parallelism plans for both phases. LLM applications often emphasize individual latency for each phase: time to first token (TTFT) for the prefill phase and time per output token (TPOT) of each request for the decoding phase. In the presence of stringent latency requirements, existing systems have to prioritize one latency over the other, or over-provision compute resources to meet both.

DistServe assigns prefill and decoding computation to different GPUs, hence eliminating prefill-decoding interferences. Given the application's TTFT and TPOT requirements, DistServe co-optimizes the resource allocation and parallelism strategy *tailored* for each phase. DistServe also places the two phases according to the serving cluster's bandwidth to minimize the communication caused by disaggregation. As a result, DistServe significantly improves LLM serving performance in terms of the maximum rate that can be served within both TTFT and TPOT constraints on each GPU. Our evaluations show that on various popular LLMs, applications, and latency requirements, DistServe can serve $7.4\times$ more requests or $12.6\times$ tighter SLO, compared to state-of-the-art systems, while staying within latency constraints for $> 90\%$ of requests.

1 Introduction

Large language models (LLMs), such as GPT-4 [37], Bard [2], and LLaMA [51], represent a groundbreaking shift in generative AI. They start to reshape existing Internet services, ranging from search engines to personal assistants [4], and enable fundamentally new applications, like universal chatbots [1, 16] and programming assistants [15, 42]. Yet, these advances come with a significant challenge: processing an end-to-end LLM query can be substantially slower than a

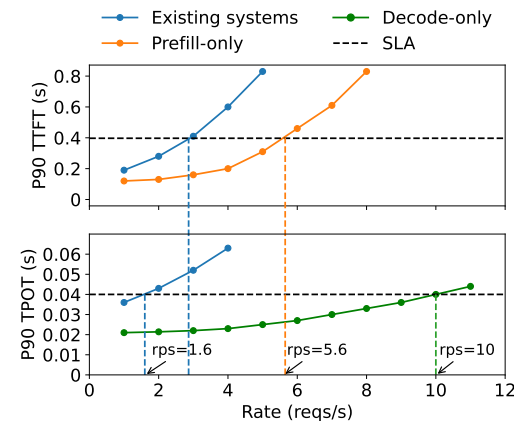


Figure 1: Performance when serving an LLM with 13B parameters under a synthetic workload with input length = 512 and output length = 64 on one NVIDIA 80GB A100. *Upper*: The P90 time-to-first-token (TTFT) latency comparing existing systems vs. a system serving only the prefill phase. *Down*: The P90 time-per-output-token (TPOT) latency comparing existing systems vs. a system serving only the decoding phase.

standard search query [41]. In order to meet the stringent latency requirements of various applications, service providers need to over-provision compute resources, particularly many GPUs, leading to a shortfall in cost efficiency. Therefore, optimizing the cost per LLM query while adhering to high *SLO attainment* (the proportion of requests that meet the SLOs) is becoming increasingly essential for all LLM services.

An LLM service responds to a user query in two phases. The *prefill phase* processes a user's prompt, composed of a sequence of tokens, to generate the first token of the response *in one step*. Following it, the *decoding phase* sequentially generates subsequent tokens *in multiple steps*; each decoding step generates a new token based on tokens generated in previous steps, until reaching a termination token. This dual-phase process distinguishes LLM services from traditional services – an LLM service's latency is uniquely measured by two key metrics: the *time to first token* (TTFT), which is the duration of the prefill phase, and the *time per output*

DistServe: 通过分离预填充和解码以优化大型语言模型服务的有效吞吐量

钟寅民¹ 刘盛宇¹ 陈俊达³ 胡建波¹ 朱亦博² 刘炫德¹ 金鑫¹ 张浩³

¹北京大学计算机科学学院 ²阶跃星辰 ³加州大学圣地亚哥分校

摘要

DistServe 通过解耦预填充和解码计算，提升了大语言模型服务性能。现有大语言模型服务系统将预填充阶段和解码阶段共置，并对所有用户和请求的预填充与解码计算进行批处理。我们发现，这种策略不仅会导致强烈的预填充-解码干扰，还会耦合两个阶段的资源分配与并行策略。大语言模型应用通常对每个阶段的单独延迟有较高要求：预填充阶段关注首字延迟（TTFT），解码阶段则关注每个请求的每输出词元时间（TPOT）。在严格的延迟要求下，现有系统必须在一种延迟与其他延迟（或过度配置计算资源以同时满足两者）之间做出取舍。

DistServe将预填充和解码计算分配给不同的GPU，从而消除了预填充-解码干扰。根据应用的TTFT和TPOT要求，DistServe针对每个阶段定制地协同优化资源分配和并行策略。此外，DistServe根据服务集群的带宽放置这两个阶段，以最小化解耦引起的通信。因此，DistServe在每块GPU上满足TTFT和TPOT约束的情况下，显著提升了大语言模型服务性能，即能够支持的最大速率。我们的评估表明，在各种流行的大语言模型、应用和延迟要求下，与最先进系统相比，DistServe可以在满足 $> 90\%$ 请求的延迟约束的前提下，处理 $7.4\times$ 倍的请求或支持 $12.6\times$ 更严格的SLO。

1 引言

大型语言模型（如GPT-4 [37]、Bard [2]和LLaMA [51]）代表了生成式人工智能的一次划时代转变。它们开始重塑现有的互联网服务，从搜索引擎到个人助理 [4]，并促成了全新的应用，例如通用聊天机器人 [1, 16] 和编程助手 [15, 42]。然而，这些进展也带来了重大挑战：处理一个端到端的大语言模型查询可能会比

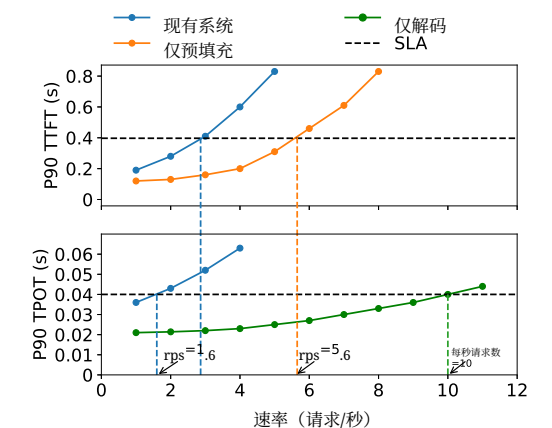


图1：在单张NVIDIA 80GB A100上，针对输入长度 = 512 和输出长度 = 64 的合成工作负载，服务130亿参数大语言模型时的性能。上图：现有系统与仅服务预填充阶段的系统在P90首token时延（TTFT）延迟方面的对比。下图：现有系统与仅服务解码阶段的系统在P90每输出token时延（TPOT）延迟方面的对比。

标准搜索查询 [41]。为了满足各种应用严格的延迟要求，服务提供商需要过度配置计算资源，特别是大量的GPU，从而导致成本效益不足。因此，在遵循高SLO达成率（满足服务等级目标的请求比例）的同时优化每个大语言模型查询的成本，对所有大语言模型服务而言正变得越来越重要。

大语言模型服务通过两个阶段响应用户查询。预填充阶段处理由一系列令牌组成的用户提示词，并在一个步骤中生成响应中的首个令牌。随后，解码阶段通过多个步骤依次生成后续令牌；每个解码步骤都基于之前步骤生成的令牌生成一个新令牌，直至达到终止令牌。这种双阶段过程使大语言模型服务区别于传统服务——大语言模型服务的延迟由两个关键指标唯一衡量：首字延迟（TTFT），即预填充阶段的持续时间，以及每个输出的



token (TPOT), which represents the average time taken to generate a token for each request (except for the first token)¹. Different applications place varying demands on each metric. For example, real-time chatbots [1] prioritize low TTFT for response promptness, while TPOT only remains important until it is faster than human reading speed (i.e., 250 words/min). Conversely, document summarization emphasizes low TPOT for faster generation of the summary.

Hence, given the application’s TTFT and TPOT requirements, an effective LLM serving system should balance these needs and maximize *per-GPU goodput*, defined as the maximum request rate that can be served adhering to the SLO attainment goal (say, 90%) for each GPU provisioned – higher per-GPU goodput directly translates into lower cost per query.

As the prefill and decoding phases share the LLM weights and working memory, existing LLM serving systems typically colocate both phases on GPUs and maximize the overall system throughput – tokens generated per second across all users and requests – by batching the prefill and decoding steps across requests [31, 54]. However, to meet latency requirements, we find these systems must over-provision compute resources. To see this, Figure 1 illustrates how the P90 TTFT and TPOT shift with increasing request rates when serving a 13B LLM using existing systems [32], with workload pattern and two latency constraints set to emulate using LLM to generate a short summary for an article. Under the SLO attainment of 90%, the maximum achievable goodput on a single A100 GPU, which is constrained by the more stringent one of TTFT and TPOT requirements, is about 1.6 requests per second (rps). The performance contrasts sharply when each phase is served independently on a separate GPU, shown by the orange and green curves, which achieve per-GPU goodput of 5.6 rps for the prefill phase and 10 rps for decoding. Ideally, by allocating 2 GPUs for prefill and 1 GPU for decoding, we can effectively serve the model with an overall goodput of 10 rps, or equally 3.3 rps per GPU, which is 2.1x higher than existing systems. The gap in goodput primarily stems from the colocation of the prefill and decoding – two phases with very distinct computational characteristics and latency requirements (§2.1).

First, colocation leads to strong *prefill-decoding interference*. A prefill step often takes much longer than a decoding step. When batched together, decoding steps in the batch are delayed by the prefill steps, significantly elongating their TPOT; similarly, the inclusion of decoding steps contributes to a non-trivial increase in TTFT, as evidenced in Figure 2. Even if we schedule them separately, issues persist as they begin to compete for resources. Decoding tasks awaiting GPU execution are subject to increased queuing delays due to ongoing prefill tasks, and vice versa. Prioritized scheduling of one phase risks failing the latency requirements of the other.

Second, the prefill and decoding computation differ in la-

tency requirements and preference for different forms of parallelism (§3). Colocating prefill and decoding, however, couples their resource allocation, and prevents implementing different parallelism strategies more suited to meeting the specific latency requirements of each phase.

To overcome these challenges, we propose to disaggregate the prefill and decoding phases of LLM inference, assigning them to separate GPUs. Our approach has two benefits. First, operating each phase independently on different GPUs eliminates prefill-decoding interference. Second, it allows to scale each phase independently with tailored resource allocation and model parallelism strategies to meet their specific latency requirements. Although disaggregation causes communication of intermediate states between GPUs, we show that the communication overhead is insubstantial (§3.3) in modern GPU clusters, and when managed appropriately, disaggregation significantly improves per-GPU goodput.

Based on the above insights, in this work, we build DistServe², a goodput-optimized LLM serving system by disaggregating the prefill and decoding phases. Given TTFT and TPOT requirements, DistServe first scales each phase independently by co-optimizing the GPU allocation and parallelism strategies of the prefill and decoding phase assuming serving a single model replica. The optimization ensures maximizing the per-GPU goodput and may assign different numbers of GPUs and parallelism strategies to each phase depending on their respective latency requirements. DistServe then scales this allocation to multiple instances via replication until meeting the user-required traffic rate (§4). DistServe also features an algorithm to place the prefill and decoding computation according to their allocation schemes and the cluster’s bandwidth to minimize the overhead of communicating intermediate states between phases.

We implement DistServe as an orchestration layer on top of the LLM inference engine. We evaluate DistServe on various LLMs, varying the workloads based on three important real-world LLM applications: chatbots, programming assistant, and document summary. Compared to state-of-the-art solutions, DistServe can serve up to $7.4\times$ more requests or $12.6\times$ tighter SLO under various latency constraints. Our contributions are:

- Identify the problems of prefill-decoding interference and resource coupling in existing LLM serving systems and propose to disaggregate the two phases.
- Design a novel placement algorithm to choose the goodput-optimal schema for prefill and decoding instances automatically.
- Conduct a comprehensive evaluation of DistServe with realistic workloads.

2 Background and Motivation

An LLM service follows a client-server architecture: the client submits a sequence of text as a request to the server; the server

令牌 (TPOT), 表示为每个请求生成一个令牌所需的平均时间 (首个令牌除外)¹。不同的应用对各项指标有不同的要求。例如, 实时聊天机器人 [1]优先考虑低 TTFT以确保响应的及时性, 而TPOT仅在快于人类阅读速度 (即每分钟250个单词) 时仍具有重要意义。相反, 文档摘要生成则强调低TPOT, 以实现更快的摘要生成。

因此, 鉴于应用的 TTFT 和 TPOT 延迟要求, 有效的大语言模型服务系统应平衡这些需求并最大化每 GPU 有效吞吐量, 即在每个配置的 GPU 上, 满足 SLO 达成目标 (例如, 90%) 时可服务的最大请求速率——更高的每 GPU 有效吞吐量直接转化为更低的每次查询成本。

由于预填充阶段和解码阶段共享大语言模型权重和工作内存, 现有大语言模型服务系统通常将这两个阶段共置在 GPU 上, 并通过跨请求批处理预填充和解码步骤来最大化整体系统吞吐量——即跨所有用户和请求生成的每秒令牌数 [31, 54]。然而, 为了满足延迟要求, 我们发现这些系统必须过度配置计算资源。为了说明这一点, 图 1 展示了在使用现有系统 [32].服务 13B 大语言模型时, 随着请求速率的增加, P90 TTFT 和 TPOT 的变化情况, 其中工作负载模式和两个延迟约束旨在模拟使用大语言模型为文章生成简短摘要的场景。在 90% 的 SLO 达成率下, 受 TTFT 和 TPOT 要求中更严格一项约束的单块 A100 GPU 上可实现的最高有效吞吐量约为 1.6 每秒请求数 (rps)。当每个阶段在单独的 GPU 上独立服务时, 性能形成鲜明对比, 如橙色和绿色曲线所示, 预填充阶段的每 GPU 有效吞吐量达到 5.6 rps, 解码阶段达到 10 rps。理想情况下, 通过为预填充分配 2 块 GPU 和为解码分配 1 块 GPU, 我们可以有效地以整体 10 rps 的有效吞吐量服务该模型, 或等效于每 GPU 3.3 rps, 比现有系统高 2.1 倍。有效吞吐量的差距主要源于预填充和解码的共置——这两个阶段具有截然不同的计算特征和延迟要求 (§2.1)。

首先, 共置会导致强烈的预填充-解码干扰。一个预填充步骤的耗时通常远长于一个解码步骤。当它们被批处理在一起时, 批处理中的解码步骤会因预填充步骤而延迟, 导致其TPOT显著延长; 同样, 解码步骤的引入也会导致 TTFT出现不可忽略的增加, 如图2所示。即使我们将它们分开调度, 问题依然存在, 因为它们开始争夺资源。等待 GPU执行的解码任务会因正在进行的预填充任务而遭受增加的排队延迟, 反之亦然。对某一阶段进行优先级调度, 可能会导致另一阶段无法满足延迟要求。

其次, 预填充和解码计算在延

¹整体请求延迟等于TTFT加上TPOT乘以解码阶段中生成的令牌数。

迟要求以及对不同形式并行性的偏好上存在差异 (§3)。然而, 将预填充和解码共置会耦合它们的资源分配, 从而阻碍实施更适合满足每个阶段特定延迟要求的不同并行策略。

为了克服这些挑战, 我们提出将大语言模型推理的预填充和解码阶段解耦, 并将它们分配给不同的GPU。我们的方法具有两大优势。首先, 在不同GPU上独立运行每个阶段, 消除了预填充-解码干扰。其次, 该方法允许通过定制的资源分配和并行策略独立扩展每个阶段, 以满足其特定的延迟要求。虽然解耦会导致GPU之间通信中间状态, 但我们证明, 在现代GPU集群中, 这种通信开销微乎其微 (§3.3), 并且在适当管理的情况下, 解耦能够显著提升每GPU有效吞吐量。

基于上述洞见, 本工作构建了DistServe², 这是一个通过分离预填充和解码阶段实现的、面向有效吞吐量优化的大语言模型服务系统。在给定TTFT和TPOT要求的前提下, DistServe首先在假设服务单个模型副本的条件下, 通过联合优化预填充阶段和解码阶段的GPU分配与并行策略, 对每个阶段进行独立扩展。该优化过程旨在最大化每 GPU有效吞吐量, 并根据各阶段的延迟要求, 可能为不同阶段分配不同数量的GPU和并行策略。随后, DistServe通过复制将该分配方案扩展至多个实例, 直至满足用户所需的流量速率 (§4)。此外, DistServe还包含一种算法, 用于根据预填充和解码的计算分配方案以及集群带宽来放置这些计算任务, 从而最小化阶段间传输中间状态时的通信开销。

我们将DistServe实现为位于大语言模型推理引擎之上的编排层。我们在多个大语言模型上对DistServe进行了评估, 并根据三种重要的现实世界大语言模型应用——聊天机器人、编程助手和文档摘要——调整了工作负载。与最先进的解决方案相比, 在多种延迟约束下, DistServe可以处理多达 $7.4\times$ 的更多请求, 或在 $12.6\times$ 的更严格SLO下提供服务。我们的贡献如下:

- 识别现有大语言模型服务系统中预填充-解码干扰和资源耦合的问题, 并提出将这两个阶段解耦。
- 设计一种新颖的放置算法, 自动选择针对预填充和解码实例的有效吞吐量最优方案。
- 对DistServe在真实工作负载下进行综合评估。

2 背景与动机

大语言模型服务遵循客户端-服务器架构: 客户端将一段文本序列作为请求提交给服务器; 服务器

²<https://github.com/LLMServe/DistServe>

¹The overall request latency equals TTFT plus TPOT times the number of generated tokens in the decoding phase.

²<https://github.com/LLMServe/DistServe>

hosts the LLM on GPUs, runs inference over the request, and responds (or streams) the generation back to the client. As explained in §1, due to the unique prefill-decoding process, LLM service may impose aggressive service-level objectives (SLOs) on both TTFT and TPOT, varying with the application’s needs. The serving system must meet both SLOs while minimizing the cost associated with expensive GPUs. In other words, we want the serving system to maximize the requests served per second adhering to the SLO attainment goal for each GPU provisioned – *maximizing per-GPU goodput*. Next, we detail the LLM inference computation (§2.1) and discuss existing optimizations for LLM serving (§2.2).

2.1 LLM Inference

Modern LLMs [37, 51] predict the next token given an input sequence. This prediction involves computing a hidden representation for each token within the sequence. An LLM can take a variable number of input tokens and compute their hidden representations in parallel, and its computation workload increases superlinearly with the number of tokens processed in parallel. Regardless of the input token count, the computation demands substantial I/O to move LLM weights and intermediate states from the GPU’s HBM to SRAM. This process is consistent across varying input sizes.

The prefill step deals with a new sequence, often comprising many tokens, and processes these tokens concurrently. Unlike prefill, each decoding step only processes one new token generated by the previous step. This leads to significant computational differences between the two phases. When dealing with user prompts that are not brief, the prefill step tends to be compute-bound. For instance, for a 13B LLM, computing the prefill of a 512-token sequence makes an A100 near compute-bound (see §3.1). In contrast, despite processing only one new token per step, the decoding phase incurs a similar level of I/O to the prefill phase, making it constrained by the GPU’s memory bandwidth.

During both phases, intermediate states, known as KV caches [32], are generated at each token position, which are needed again in later decoding steps. To avoid recomputing them, they are saved in GPU memory. Because of the shared use of LLM weights and KV caches in memory, most LLM inference engines opt to colocate the prefill and decoding phases on GPUs, despite their distinct computational characteristics.

2.2 LLM Serving Optimization

In real-time online serving, multiple requests come and must be served within SLOs. Batching and parallelizing their computation is key for achieving low latency, high throughput, and high utilization of GPUs.

Batching. Current serving systems [9, 32, 54] utilize a batching technique known as *continuous batching*. This method batches the prefill of new requests with the decoding of ongoing ones. It boosts the GPU utilization and maximizes the overall system throughput – tokens generated per second

across all users and requests. However, as mentioned in §1 and elaborated later in §2.3, this approach leads to trade-offs between TTFT and TPOT. An advanced variant of continuous batching [9] attempts to balance TTFT and TPOT by segmenting long prefill into chunks and attaching decoding jobs with a chunked prefill – but essentially, it trades TTFT for TPOT and cannot eliminate the interference (§2.3). In summary, batching prefill and decoding invariably leads to compromises in either TTFT or TPOT.

Model parallelism. In LLM serving, model parallelism is generally divided as intra- and inter-operator parallelisms [33, 46, 59]. Both can be used to support larger models but may impact serving performance differently. Intra-operator parallelism partitions computationally intensive operators, such as matrix multiplications, across multiple GPUs, accelerating computation but causing substantial communication. It reduces the execution time³, hence latency, particularly for TTFT of the prefill phase, but requires high bandwidth connectivity between GPUs (e.g., NVLINK). Inter-operator parallelism organizes LLM layers into stages, each running on a GPU to form pipelines. It moderately increases execution time due to inter-stage communication, but linearly scales the system’s rate capacity with each added GPU. In this paper, we reveal an additional benefit of model parallelism: reduced queuing delay of both prefill and decoding phases, steaming from shorter execution time. We delve into this further in §3. Besides model parallelism, replicating a model instance, irrespective of its model parallelism configurations, linearly scales the system’s rate capacity.

These parallelism strategies create a complex space of optimization that requires careful trade-offs based on the application’s latency requirements.

2.3 Problems and Opportunities

Colocating and batching the prefill and decoding computation to maximize the overall system throughput, as in existing systems, is cost-effective for service providers. However, in the presence of SLOs, present approaches struggle to maintain both high service quality and low cost due to the issues discussed below.

Prefill-decoding interference. As Figure 2 shows, adding a single prefill job to a batch of decoding requests significantly slows down both processes, leading to a marked increase in TTFT and TPOT. Specifically, the decoding tasks in the batch must wait for lengthier prefill jobs to complete, thus extending TPOT; the slowdown intensifies with a longer prefill, shown in Figure 2(b). Adding decoding jobs to prefill also increases the time to complete the prefill task, particularly when the GPU is already at capacity (Figure 2 blue curves).

One attempt to mitigate this interference is called *chunked-prefill with piggyback* [3, 9]. It proposes to split the long prefill

³we emphasize “execution time” instead of latency here because latency comprises both execution time and queuing delay.

在GPU上托管大语言模型，对请求执行推理，并将生成结果返回（或流式传输）给客户端。如§1所述，由于预填充-解码过程的独特性，大语言模型服务可能对TTFT和TPOT都施加严格的服务等级目标，具体随应用需求而变化。服务系统在满足两项服务等级目标的同时，必须最小化昂贵GPU相关的成本。换言之，我们希望服务系统能够最大化每秒处理且符合每个服务等级目标达成目标的请求数——即最大化每GPU有效吞吐量。接下来，我们将详细介绍大语言模型推理的计算（§2.1），并讨论现有针对大语言模型服务的优化（§2.2）。

2.1 LLM推理

现代大语言模型 [37, 51] 根据输入序列预测下一个令牌。该预测涉及计算序列中每个令牌的隐藏表示。大语言模型可以接收可变数量的输入令牌，并并行计算它们的隐藏表示，且其计算工作负载随着并行处理的令牌数量呈超线性增长。无论输入令牌数量如何，计算过程都需要大量的输入/输出来将大语言模型权重和中间状态从GPU的HBM移动到SRAM。该过程在不同的输入规模下保持一致。

预填充步骤处理新序列，通常包含许多令牌，并并行处理这些令牌。与预填充不同，每个解码步骤仅处理前一步生成的一个新令牌。这导致两个阶段之间存在显著的计算差异。当处理较长的用户提示时，预填充步骤往往受限于计算。例如，对于13B大语言模型，计算512个令牌序列的预填充会使A100接近计算受限状态（见§3.1）。相比之下，尽管每个步骤仅处理一个新令牌，解码阶段仍会产生与预填充阶段相似水平的输入/输出，使其受限于GPU的内存带宽。

在这两个阶段中，在每一个令牌位置都会生成中间状态，即KV缓存 [32]，而这些KV缓存存在后续解码步骤中会被再次使用。为避免重新计算，它们会被保存在GPU内存中。由于在内存中共享使用大语言模型权重和KV缓存，大多数大语言模型推理引擎选择在GPU上共置预填充阶段和解码阶段，尽管这两者具有不同的计算特性。

2.2 大语言模型服务优化

在实时在线服务中，多个请求陆续到达，且必须在服务等级目标（SLOs）内得到处理。对这些请求的计算进行批处理和并行化是实现低延迟、高吞吐量以及高GPU利用率的关键。

批处理。当前的服务系统 [9, 32, 54] 使用一种称为连续批处理的技术。该方法将新请求的预填充与正在进行的请求的解码进行批处理。它提高了GPU利用率，并最大化了整个系统的吞吐量——每秒生成的令牌数

，涵盖所有用户和请求。然而，如§1所述并在§2.3中详细阐述，这种方法会导致TTFT和TPOT之间的权衡。连续批处理的一种高级变体 [9] 通过将长预填充分割成多个分块，并将解码任务与分块预填充相结合，试图在TTFT和TPOT之间取得平衡——但本质上，它是以TTFT换取TPOT，无法消除干扰（§2.3）。总而言之，将预填充和解码进行批处理不可避免地会在TTFT或TPOT中做出妥协。

模型并行。在大语言模型服务中，模型并行通常分为算子内并行和算子间并行 [33, 46, 59]。这两种方式都可以用于支持更大的模型，但会对服务性能产生不同的影响。算子内并行将计算密集型算子（如矩阵乘法）划分到多个GPU上，从而加速计算，但会带来大量的通信。它能减少执行时间³，进而降低延迟，特别是预填充阶段的TTFT，但这需要GPU之间具有高带宽连接（例如NVLink）。算子间并行将大语言模型的层组织成阶段，每个阶段在一个GPU上运行以形成流水线。由于阶段间通信，它会适度增加执行时间，但系统速率容量会随着每个新增的GPU线性扩展。在本文中，我们揭示了模型并行的另一项优势：预填充阶段和解码阶段的排队延迟降低，这源于更短的执行时间。我们将在第3节中深入探讨这一点。除了模型并行之外，无论模型并行配置如何，复制模型实例都能使系统的速率容量线性扩展。

这些并行策略构建了一个复杂的优化空间，需要根据应用的延迟要求进行谨慎的权衡。

2.3 问题与机遇

如现有系统那样，将预填充和解码计算进行共置和批处理以最大化整体系统吞吐量，对服务提供商而言是具有成本效益的。然而，在服务等级目标（SLOs）存在的背景下，由于下述问题，现有方法难以同时保持高质量和低成本。

预填充-解码干扰。如图2所示，将一个预填充任务加入解码请求的批处理中，会显著减缓这两个过程，导致TTFT和TPOT显著增加。具体而言，批处理中的解码任务必须等待耗时更长的预填充任务完成，从而延长了TPOT；随着预填充时间变长，减速效应加剧，如图2(b)所示。将解码任务加入预填充也会增加完成预填充任务所需的时间，尤其是在GPU已满载时（图2中的蓝色曲线）。

一种缓解这种干扰的方法被称为带搭载的块状预填充 [3, 9]。它建议将长的预填充

³此处我们强调“执行时间”而非延迟，因为延迟同时包含执行时间和排队延迟。

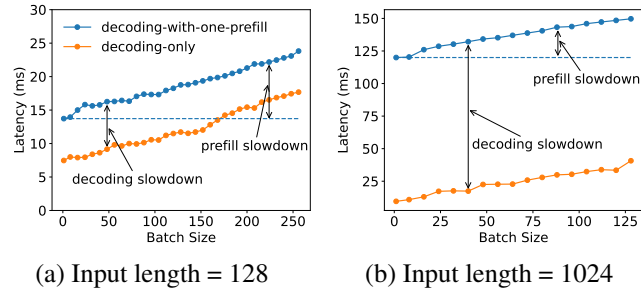


Figure 2: Batch execution time when serving a 13B LLM as batch size increases. Compared between a decoding-only batch and the batch adding one more prefill job.

into chunks and batch a prefill chunk with a few decoding jobs (a.k.a. piggybacking). This technique alleviates the slowdown of the decoding job caused by the long prefill job, but it does not eliminate it. Additionally, it results in an extra overhead for the prefill job which cannot be easily mitigated by adjusting the chunk size. First, if the chunk size is set much lower than the inflection point that can saturate the GPU, then the prefill job will have a longer execution time since it competes with the decoding job in the same batch and cannot solely utilize the GPU resources. Second, if we increase the chunk size to nearly saturate the GPU, the chance of piggybacking will diminish since the remaining slots for decode tokens are limited. Also, chunked-prefill causes significantly more memory access for the prefill jobs. This is because the KV cache of all previous chunks have to be loaded from HBM to SRAM repeatedly to compute each subsequent chunk. Concretely, if a prefill job is split into N equal chunks, we need to load $N + (N - 1) + \dots + 1 = O(N^2)$ chunks of KV Cache in total, compared to $O(N)$ in the non-chunked case. This overhead will increase as the context length becomes longer.

Ineffective scheduling. Unbatching prefill and decoding jobs and scheduling them sequentially does not mitigate the interference. Decoding jobs may experience longer queuing delays due to waiting for ongoing prefill jobs on GPUs. Moreover, batches dedicated to decoding often lead to GPU underutilization. Prioritizing tasks in either phase adversely affects the latency of the other, rendering priority scheduling ineffective.

Resource and parallelism coupling. Colocating prefill and decoding phases on the same GPUs unavoidably share their resource and parallelism settings. However, each phase has its unique computational characteristic and latency requirement that calls for more heterogeneous resource allocation. For example, the prefill phase tends to be compute-bound and benefits from more intra-op parallelism to reduce execution time to meet the tight SLO on TTFT. By contrast, the optimal parallelism configuration of the decoding phase depends on the running batch size. In existing systems, due to coupling, resource allocation and parallelism plans are tailored to satisfy the *more demanding* of TTFT and TPOT, which may not be ideal for the other. This often leads to resource over-provisioning to meet both SLOs.

Opportunities. To address these issues, we propose to disaggregate the prefill and decoding phases. We use the term *instance* to denote a unit of resources that manages exactly one complete copy of model weights. One instance can correspond to many GPUs when model parallelism is applied. Note that when we disaggregate the two phases to different GPUs, each phase manages its copy of the model weights, resulting in *prefill instances* and *decoding instances*. A prefill instance, upon receiving a request, performs only the prefill computation for this request to generate the first output token. It then sends the intermediate results (mainly KV caches) to a decoding instance, which is responsible for subsequent decoding steps. Because decoding computation often has low GPU utilization, we may allocate multiple prefill instances per decoding instance. This allows batching more decoding jobs to achieve higher GPU utilization.

Disaggregating prefill and decoding naturally resolves the interference between the two phases and enables each to focus on its optimization target – TTFT or TPOT. Each type of instance can employ different resources and parallelism strategies to meet a variety of latency requirements. By adjusting the number of GPUs and parallelisms provided to the two types of instances, we can maximize the per-device goodput of the overall system, avoiding over-provisioning, eventually translating to reduced cost-per-query adhering to service quality. Next, we develop ways to find out the best resource allocation and parallelism plan for each phase.

3 Tradeoff Analysis

Disaggregation uncouples the two phases and allows a distinct analysis of the characteristics of each phase, providing valuable insights into the algorithm design. It also expands the design space: now each phase needs to be scaled and scheduled independently based on their latency requirements.

In this section, we analyze the computational pattern of prefill (§3.1) and decoding instances (§3.2) *post disaggregation*. We aim to identify key parameters and derive guidelines for batching and parallelism in each phase. We then highlight several practical deployment considerations (§3.3). This section lays the foundation for per-gpu goodput optimization.

3.1 Analysis for Prefill Instance

After disaggregation, the prefill phase generates the first token by processing all tokens of the user prompt in parallel. Assuming a given arrival rate, we aim to fulfill the service’s latency requirement on TTFT using the least resources.

Batching strategy. The prefill step is typically compute-intensive. Figure 3(a) shows how the throughput of the prefill phase changes with the input length and the batch size. For a 13B parameter LLM, processing a single sequence of 512 tokens can fully engage an A100 GPU. Once the GPU becomes compute-bound, adding more requests to the batch no longer improves GPU efficiency. Instead, it proportionally extends the total processing time for the batch, inadvertently

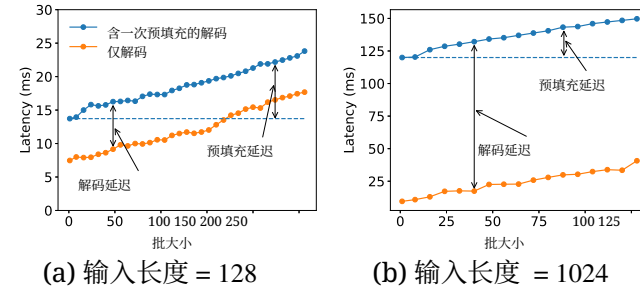


Figure 2: 随着批大小增加，为13B大语言模型提供服务时的批处理执行时间。比较了仅解码批处理与额外添加一个预填充任务的批处理。

将请求分割为分块，并将一个预填充分块与少量解码任务（即 piggybacking）组成一个批处理。这种技术缓解了长预填充任务对解码任务造成的减速影响，但无法将其消除。此外，它还会为预填充任务带来额外的开销，而通过调整分块大小难以轻易缓解。首先，如果将分块大小设置得远低于能使 GPU 饱和的拐点，由于预填充任务在同一批处理中与解码任务竞争，且无法独占 GPU 资源，其执行时间将会变长。其次，如果我们增加分块大小以近乎饱和 GPU，由于剩余用于解码令牌的槽位有限，piggybacking 的机会将会减少。此外，分块预填充会导致预填充任务的内存访问显著增加。这是因为，为了计算每个后续分块，必须将所有先前分块的 KV 缓存反复从 HBM 加载到 SRAM。具体而言，如果一个预填充任务被拆分为 N 个等长分块，与不分块情况下的 $O(N)$ 相比，总共需要加载 $N + (N - 1) + \dots + 1 = O(N^2)$ 个分块的 KV 缓存。随着上下文长度增加，这种开销将会增大。

低效的调度。将预填充和解码任务拆分为非批处理并按顺序调度，并不能缓解干扰。由于等待GPU上正在进行的预填充任务，解码任务可能会经历更长的排队延迟。此外，专门用于解码的批处理往往导致GPU利用率不足。优先考虑任一阶段的任务都会对另一阶段的延迟产生不利影响，从而使优先级调度无效。

资源与并行度耦合。将预填充和解码阶段部署在同一组 GPU 上，不可避免地会共享其资源与并行度配置。然而，每个阶段都具有独特的计算特征和延迟要求，因此需要更加异构的资源分配。例如，预填充阶段通常是计算受限的，并受益于更多的算子内并行性，从而缩短执行时间，以满足TTFT上严格的服务等级目标（SLO）。相比之下，解码阶段的最佳并行性配置取决于正在运行的批大小。在现有系统中，由于耦合关系，资源分配和并行策略计划是根据TTFT和TPOT中要求更严格的一方来调整的，这对另一方而言可能并非理想选择。这通常会导致为了满足双方的服务等级目标而进行资源过度配置。

机会。为了解决这些问题，我们提出解耦预填充和解码阶段。我们使用术语“实例”来表示管理恰好一份完整模型权重副本的资源单元。当应用模型并行时，一个实例可以对应多个GPU。请注意，当我们把这两个阶段解耦到不同的GPU上时，每个阶段管理其自身的模型权重副本，从而形成预填充实例和解码实例。预填充实例在收到请求后，仅执行该请求的预填充计算以生成第一个输出令牌。随后，它将中间结果（主要是KV缓存）发送到一个解码实例，该实例负责后续的解码步骤。由于解码计算通常具有较低的GPU利用率，我们可以为每个解码实例分配多个预填充实例。这使得能够批处理更多的解码任务，从而实现更高的GPU利用率。

将预填充和解码分离自然解决了两个阶段之间的干扰，并使每个阶段能够专注于其优化目标——TTFT或TPOT。每种类型的实例可以针对不同资源分配和并行策略来满足各种延迟要求。通过调整提供给两种类型实例的GPU数量和并行化程度，我们可以最大化整个系统的每设备有效吞吐量，避免资源过度配置，最终在遵守服务质量的前提下降低每查询成本。接下来，我们将研究如何为每个阶段找出最佳的资源分配和并行策略计划。

3 权衡分析

解耦将两个阶段分离，使得能够独立分析每个阶段的特性，为算法设计提供了宝贵的洞察。它同时扩展了设计空间：现在，每个阶段都需要根据其延迟要求独立进行扩展和调度。

在本节中，我们分析了解耦后预填充 (§3.1) 和解码实例 (§3.2) 的计算模式。我们的目标是识别关键参数，并推导各阶段中批处理和并行性的指导原则。随后，我们强调了若干实际的部署考量 (§3.3)。本节为每 GPU 有效吞吐量优化奠定了基础。

3.1 预填充实例分析

解耦后，预填充阶段通过并行处理用户提示中的所有令牌来生成首个令牌。在给定到达率的前提下，我们的目标是以最少资源满足服务对TTFT的延迟要求。

批处理策略。预填充步骤通常是计算密集型的。图3(a)展示了预填充阶段的吞吐量随输入长度和批大小变化的情况。对于一个13B参数的大语言模型，处理一个包含512个令牌的单个序列即可充分利用A100 GPU。一旦GPU进入计算受限状态，向批处理中添加更多请求不再提高GPU效率，反而会按比例延长该批次的总处理时间，无意中

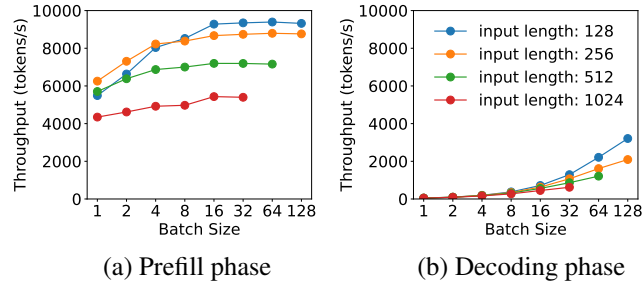


Figure 3: Throughput for two phases with different batch sizes and input lengths when serving an LLM with 13B parameters.

delaying all included requests. Hence, for prefill instances, it is necessary to profile the specific LLM and GPUs in advance to identify a critical input length threshold, denoted as L_m , beyond which the prefill phase becomes compute-bound. Batching more requests should only be considered when the input length of the scheduled request is below L_m . In practice, user prompts typically average over hundreds of tokens [8]. Batch sizes for the prefill instance are generally kept small.

Parallelism plan. To study the parallelism preferences for prefill-only instances, we serve a 66B LLM on two A100 GPUs with inter-op or intra-op parallelism strategy. To simplify the problem, we assume uniform requests input lengths of 512 tokens and a Poisson arrival process. We compare the resulting average TTFT at various arrival rates in Figure 4(a): intra-op parallelism is more efficient at lower arrival rates, while inter-op parallelism gains superiority as the rate increases. Disaggregation enables the prefill phase to function analogously to an M/D/1 queue, so we can use queuing theory to verify the observation.

We start by developing notations using the single-device case without parallelism: each request’s execution time, denoted as D , remains constant due to uniform prefill length. Since one request saturates the GPU, we schedule requests via First-Come-First-Served (FCFS) without batching. Suppose the Poisson arrival rate is R and the utilization condition of $RD < 1$, the average TTFT (Avg_TTFT) can be modeled by the M/D/1 queue [47] in close form:

$$Avg_TTFT = D + \frac{RD^2}{2(1-RD)}. \quad (1)$$

where the first term represents the execution time and the second corresponds to the queuing delay. Based on Eq. 1, we incorporate parallelism below.

With 2-way inter-op parallelism, we assume the request-level latency becomes D_s , and the slowest stage takes D_m to finish. We have $D \approx D_s \approx 2 \times D_m$, due to negligible inter-layer activation communication [33, 59]. The average TTFT with 2-way inter-op parallelism is derived as:

$$Avg_TTFT_{inter} = D_s + \frac{RD_m^2}{2(1-RD_m)} = D + \frac{RD^2}{4(2-RD)}. \quad (2)$$

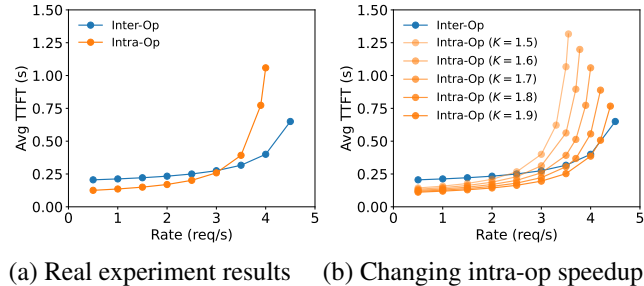


Figure 4: Average TTFT when serving an LLM with 66B parameters using different parallelism on two A100 GPUs.

For intra-op parallelism, we introduce a speedup coefficient K , where $1 < K < 2$, reflecting the imperfect speedup caused by high communication overheads of intra-op parallelism. With the execution time $D_s = \frac{D}{K}$, the average TTFT for 2-degree intra-op parallelism is:

$$Avg_TTFT_{intra} = \frac{D}{K} + \frac{RD^2}{2K(K-RD)}. \quad (3)$$

Comparing Eq. 2 and Eq. 3: at lower rates, where execution time (first term) is the primary factor, intra-op parallelism’s reduction in execution time makes it more efficient. As the rate increases and the queuing delay (second term) becomes more significant, inter-op parallelism becomes advantageous, concurred with Figure 4(a).

The prefill phase’s preference for parallelism is also influenced by TTFT SLO and the speedup coefficient K . Seen from Figure 4(a): A more stringent SLO will make intra-op parallelism more advantageous, due to its ability to reduce execution time. The value of K depends on factors such as the input length, model architecture, communication bandwidth, and placement [46, 59]. As shown in Figure 4(b), a decrease in K notably reduces the efficacy of intra-op parallelism. §4 develops algorithms that optimize the resource and parallelism configurations taking into consideration these knobs.

3.2 Analysis for Decoding Instance

Unlike the prefill instance, a decoding instance follows a distinct computational pattern: it receives the KV caches and the first output token from the prefill instance and generates subsequent tokens one at a time. For decoding instances, our optimization goal is to satisfy the application’s TPOT requirement using minimal computing resources.

Batching strategy. Since a single decoding job is heavily bandwidth-bound, batching is key to avoiding low GPU utilization (hence high per-gpu goodput), as shown in Figure 3(b). In existing systems where the prefill and decoding phases are colocated, increasing the decoding batch size is difficult because it conflicts with meeting latency goals, particularly in scenarios with high request rates. This is because sharing GPUs cause competition between prefill and decoding jobs, leading to a trade-off between TTFT and TPOT. For

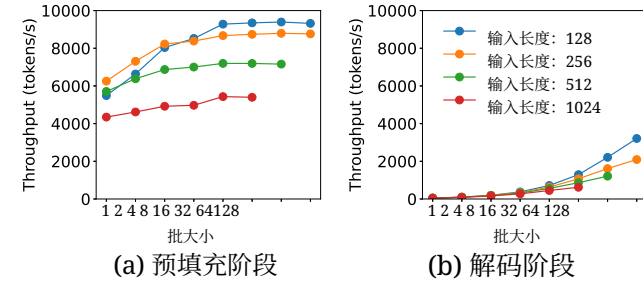


图 3: 在提供具有 130 亿参数的大语言模型服务时, 两个阶段在不同批次大小和输入长度下的吞吐量。

延迟所有包含的请求。因此, 对于预填充实例, 需要提前对特定大语言模型和GPU进行性能分析, 以识别一个关键的输入长度阈值, 记为 L_m , 超过该阈值后, 预填充阶段将变为计算受限。仅当调度请求的输入长度低于 L_m 时, 才应考虑增加更多请求进行批处理。在实践中, 用户提示的平均长度通常达到数百个令牌 [8]。预填充实例的批大小通常保持较小。

并行策略计划。为了研究仅预填充实例的并行性偏好, 我们在两个 A100 GPU 上使用算子间并行或算子内并行策略来提供 660 亿参数大语言模型服务。为简化问题, 我们假设均匀请求输入长度为 512 个令牌, 并采用泊松到达过程。我们在图 4(a) 中比较了在不同到达率下得到的平均 TTFT: 在较低的到达率下, 算子内并行效率更高, 而随着速率增加, 算子间并行取得优势。解耦使预填充阶段能够类似地运作为一个 M/D/1 队列, 因此我们可以使用排队论来验证这一观察。

我们首先基于无并行性的单设备情况来开发符号: 由于预填充长度均匀, 每个请求的执行时间 (记为 D) 保持恒定。由于一个请求即可使 GPU 饱和, 我们通过先到先服务 (FCFS) 来调度请求, 而不进行批处理。假设泊松到达率为 R , 且利用率条件为 $RD < 1$, 平均 TTFT (Avg_TTFT) 可以通过 M/D/1 队列 [47] 的封闭形式来建模:

$$Avg_TTFT = D + \frac{RD^2}{2(1-RD)}. \quad (1)$$

其中第一项表示执行时间, 第二项对应排队延迟。基于公式 1, 我们在下文引入了并行性。

在采用 2 路操作间并行的情况下, 我们假设请求级延迟变为 D_s , 而最慢的阶段需要 D_m 才能完成。由于层间激活通信 [33, 59] 可忽略不计, 我们有 $D \approx D_s \approx 2 \times D_m$ 。采用 2 路操作间并行时的平均 TTFT 推导如下:

$$Avg_TTFT_{inter} = D_s + \frac{RD_m^2}{2(1-RD_m)} = D + \frac{RD^2}{4(2-RD)}. \quad (2)$$

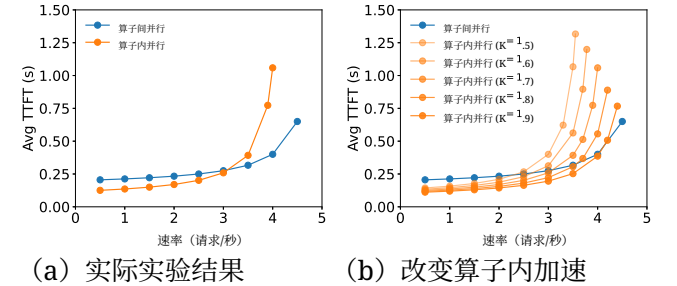


图 4: 在使用两台 A100 GPU 以不同的并行方式服务具有 66B 参数的大语言模型时的平均 TTFT。

对于算子内并行, 我们引入加速系数 K , 其中 $1 < K < 2$, 以反映算子内并行因高通信开销导致的非理想加速。在执行时间为 $D_s = \frac{D}{K}$ 的情况下, 2 路算子内并行的平均 TTFT 为:

$$Avg_TTFT_{intra} = \frac{D}{K} + \frac{RD^2}{2K(K-RD)}. \quad (3)$$

对比公式 2 和公式 3: 在较低速率下, 执行时间 (第一项) 是主要影响因素, 算子内并行通过缩短执行时间而更具效率。随着速率增加且排队延迟 (第二项) 变得更加显著, 算子间并行变得更有优势, 这与图 4(a) 的结果一致。

预填充阶段对并行性的偏好还受 TTFT SLO 和加速系数 K 的影响。从图 4(a) 可以看出: 更严格的 SLO 会使算子内并行更具优势, 因为它能够缩短执行时间。 K 的值取决于输入长度、模型架构、通信带宽和放置方案 [46, 59] 等因素。如图 4(b) 所示, K 的减小会显著降低算子内并行的效果。第 4 节开发了优化资源和并行配置的算法, 并考虑了这些调节参数。

3.2 解码实例分析

与预填充实例不同, 解码实例遵循一种不同的计算模式: 它从预填充实例接收 KV 缓存和第一个输出令牌, 并逐个生成后续令牌。对于解码实例, 我们的优化目标是在满足应用的 TPOT 需求的同时, 使用最少的计算资源。

批处理策略。由于单个解码作业严重受限于带宽, 批处理对于避免低 GPU 利用率 (从而获得较高的每 GPU 有效吞吐量) 至关重要, 如图 3(b) 所示。在预填充和解码阶段共置的现有系统中, 增加解码批大小是困难的, 因为它会与满足延迟目标相冲突, 尤其是在高请求速率的场景下。这是因为共享 GPU 会导致预填充任务和解码任务之间的竞争, 从而在 TTFT 和 TPOT 之间产生权衡。例如,

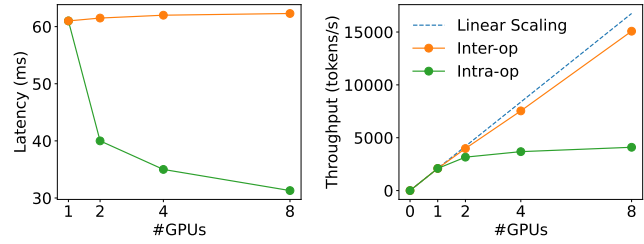


Figure 5: Decoding phase latency and throughput when serving a 13B LLM with batch size = 128 and input length = 256 under different parallel degrees.

example, a higher arrival rate generates more prefill jobs, demanding greater GPU time to meet TTFT requirements if prioritizing prefill jobs, which in turn adversely affects TPOT.

On the contrary, disaggregation offers a solution by enabling the allocation of multiple prefill instances to a single decoding instance. This approach allows for accumulating a larger batch size on dedicated GPUs for the decoding phase without sacrificing TPOT.

Parallelism plan. Post-disaggregation, the batch size for decoding may be constrained by GPU memory capacity, as it is necessary to maintain the KV caches for all active requests. Scaling the decoding instance with model parallelism or leveraging advanced memory management techniques for LLM KV caches, such as Paged-Attention [32] and GQA [10], enable further scaling of the decoding batch size to nearly compute-bound. As the decoding batch size continue to increase to approach the compute-bound, the decoding computation begins to resemble the prefill phase. With this observation, we investigate how the latency and throughput change under different parallelism degrees under large batch conditions in Figure 5: intra-op parallelism reduces latency with diminishing returns, caused by communication and reduced utilization after partitioning. Inter-op parallelism can almost linearly scale the throughput. Hence, when the TPOT SLO is stringent, intra-op parallelism is essential to reduce TPOT to meet latency goals. Beyond this, inter-op parallelism is preferable to enhance throughput linearly.

It is worth noting that when the model can fit into the memory of a single GPU, replication is a competitive option in addition to model parallelism for both prefill and decoding instances, to linearly scale the system’s rate capacity. It may also reduce the queuing delay – as indicated by Eq. 1 – by substituting R with R/N assuming requests are equally dispatched to N replicas, at the cost of maintaining additional replicas of the model weights in GPU memory.

3.3 Practical Problems

We have developed foundational principles for selecting batching and parallelisms for each phase. In this section, we discuss and address several challenges encountered during the practical deployment of disaggregated prefill and decoding phases.

Variable prefill length. §3 has assumed uniform prompt length across requests. In real deployments, depending on the LLM application, the lengths of requests are non-uniform. The non-uniformity can cause pipeline bubbles [28, 36] for prefill instances applying inter-op parallelism because the execution time of pipeline stages across requests of different lengths will vary. This results in slight deviations from the conclusions indicated by using the M/D/1 queue model. To address the problem, §4 develops algorithms that search for parallelisms based on workloads, and resort to scheduling to minimize the bubbles (§4.3).

Communication overhead. Transferring KV caches from prefill to decoding instances incurs notable overheads. For example, the KV cache size of a single 512-token request on OPT-66B is approximately 1.13GB. Assuming an average arrival rate of 10 rps, we need to transfer 11.3GB data per second—or equivalently 90Gbps bandwidth to render the overhead invisible. While many modern GPU clusters for LLMs are equipped with InfiniBand (e.g., 800 Gbps), in cases where cross-node bandwidth is limited, DistServe relies on the commonly available intra-node NVLINK, where the peak bandwidth between A100 GPUs is 600 GB/s, again rendering the transmission overhead negligible (see §6.3). However, this requirement imposes additional constraints on the placement of prefill and decoding instances that we take into consideration in the next section.

Through the analysis in this section, we identify the workload pattern, placement constraints, SLO requirements, parallelism strategies, and resource allocation as key parameters that create a web of considerations in designing the disaggregated serving system. How to automatically navigate the search space to find the configuration that achieves optimal per-gpu goodput is challenging, and addressed next.

4 Method

We built DistServe to solve the above challenges. Given the model, workload characteristic, latency requirements, and SLO attainment target, DistServe will determine (a) the parallelism strategies for prefill and decoding instances, (b) the number of each instance type to deploy, as well as (c) how to place them onto the physical cluster. We call the solution a *placement*. Our goal is to find a placement that maximizes the per-gpu goodput.

As explained in §3.3, a key design consideration is to manage communications between disaggregated prefill and decoding phases, given varying cluster setups. In this section, we first present two placement algorithms: one for clusters with high-speed cross-node networks (§4.1) and the other for environments lacking such infrastructure (§4.2); the latter introduces additional constraints. We then develop online scheduling optimizations that adapt to the nuances of real-world workloads (§4.3).

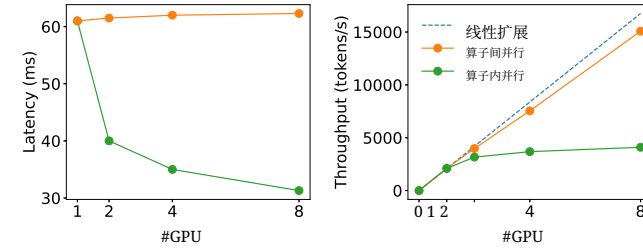


图 5：以批大小 = 128 和输入长度 = 256 服务 13B 大语言模型时，不同并行度下解码阶段的延迟和吞吐量。

较高的到达率会产生更多的预填充任务，如果优先处理预填充任务，则需要更多的GPU时间来满足TTFT要求，这反过来又会负面影响TPOT。

相反，解耦提供了一种解决方案，即允许将多个预填充实例分配给单个解码实例。这种方法可以在解码阶段的专用GPU上累积更大的批大小，而不会牺牲TPOT服务等级目标。

并行策略计划。解耦之后，解码的批大小可能受GPU内存容量的限制，因为需要为所有活动请求维护KV缓存。通过模型并行对解码实例进行扩展，或利用针对大语言模型KV缓存的先进内存管理技术（如分页注意力机制 [32] 和GQA [10]），可以进一步将解码批大小扩展到接近计算受限的水平。随着解码批大小不断增大并接近计算受限状态，解码计算开始类似于预填充阶段。基于这一观察，我们在图 5 中研究了在大批量条件下不同并行度下延迟和吞吐量的变化：算子内并行以边际递减收益的方式降低延迟，这是由于分区后的通信开销和降低的利用率所致。算子间并行可以几乎线性地扩展吞吐量。因此，当TPOT服务等级目标非常严格时，算子内并行对于降低TPOT以满足延迟目标至关重要。在此之上，算子间并行更适合线性增强吞吐量。

值得注意的是，当模型能够适配单GPU的内存时，复制是一种与模型并行具有竞争力的替代方案，可用于预填充实例和解码实例，以线性扩展系统的速率容量。通过假设请求被均匀分发到N个副本，将Eq. 1中的R替换为R/N，这也可能降低排队延迟，但其代价是在GPU内存中维护额外的模型权重副本。

3.3 实际问题

我们为选择每个阶段的批处理和并行化方法制定了基础原则。在本节中，我们讨论并解决了在分离的预填充和解码阶段的实际部署过程中遇到的若干挑战。

可变的预填充长度。第3节假设各请求的提示词长度统一。在实际部署中，由于大语言模型应用的不同，请求的长度是非均匀的。这种不均匀性会导致采用算子间并行的预填充实例出现流水线气泡 [28, 36]，因为不同长度请求的各流水线阶段的执行时间会发生变化。这会导致与使用M/D/1队列模型得出的结论出现轻微偏差。为了解决这一问题，第4节开发了基于工作负载搜索并行化方案的算法，并采用调度方法以最小化气泡（第4.3节）。

通信开销。将KV缓存从预填充实例传输到解码实例会带来显著的开销。例如，OPT-66B上单个512令牌请求的KV缓存大小约为1.13GB。假设平均到达率为10每秒请求数，则需要每秒传输11.3GB的数据——或者等效地需要90Gbps的带宽才能使开销变得可以忽略不计。虽然许多用于大语言模型的现代GPU集群配备了InfiniBand（例如，800 Gbps），但在跨节点带宽受限的情况下，DistServe依赖于普遍可用的节点内NVLink，其中A100 GPU之间的峰值带宽为600 GB/s，从而使传输开销变得可以忽略不计（参见§6.3）。然而，这一要求对预填充实例和解码实例的放置方案施加了额外的约束，我们将在下一节中考虑这一点。

通过本节分析，我们识别出工作负载模式、放置方案约束、SLO要求、并行策略和资源分配是在设计解耦式服务系统时相互交织的关键参数。如何在搜索空间中自动导航，以找到实现最优每GPU有效吞吐量的配置，是一个挑战，将在下文探讨。

4 方法

我们构建了DistServe以解决上述挑战。给定模型、工作负载特征、延迟要求和SLO达标目标，DistServe将确定：(a) 预填充和解码实例的并行策略，(b) 部署的每种实例类型的数量，以及 (c) 如何在物理集群上进行放置。我们将该解决方案称为放置方案。我们的目标是找到能最大化每GPU有效吞吐量的放置方案。

如第3.3节所述，一项关键的设计考量是在不同的集群配置下，管理解耦的预填充阶段与解码阶段之间的通信。在本节中，我们首先介绍两种放置算法：一种适用于配备高速跨节点网络的集群（第4.1节），另一种适用于缺乏此类基础设施的环境（第4.2节）；后者引入了额外的约束。随后，我们开发了针对实际工作负载细微差异进行自适应调整的在线调度优化（第4.3节）。

Algorithm 1 High Node-Affinity Placement Algorithm

Input: LLM G , #node limit per-instance N , #GPU per-node M , GPU memory capacity C , workload W , traffic rate R .

Output: the placement $best_plm$.

```
configp, configd ← 0, 0
for intra_op ∈ {1, 2, ..., M} do
  for inter_op ∈ {1, 2, ...,  $\frac{N \times M}{intra\_op}$ } do
    if  $\frac{G.size}{inter\_op \times intra\_op} < C$  then
      config ← (inter_op, intra_op)
       $\hat{G} \leftarrow \text{parallel}(G, config)$ 
      config.goodput ← simu_prefill( $\hat{G}, W$ )
      if  $\frac{config_p.goodput}{config_p.num\_gpus} < \frac{config.goodput}{config.num\_gpus}$  then
        configp ← config
      config.goodput ← simu_decode( $\hat{G}, W$ )
      if  $\frac{config_d.goodput}{config_d.num\_gpus} < \frac{config.goodput}{config.num\_gpus}$  then
        configd ← config
n, m ←  $\lceil \frac{R}{config_p.goodput} \rceil, \lceil \frac{R}{config_d.goodput} \rceil$ 
best_plm ← (n, configp, m, configd)
return best_plm
```

4.1 Placement for High Node-Affinity Cluster

On high node-affinity clusters equipped with Infiniband, KV caches transmission overhead across nodes is negligible, DistServe can deploy prefill and decoding instances across any two nodes without constraints. We propose a two-level placement algorithm for such scenarios: we first optimize the parallelism configurations for prefill and decoding instances separately to attain phase-level optimal per-gpu goodput; then, we use replication to match the overall traffic rate.

However, finding the optimal parallel configuration for a single instance type, such as for the prefill instance, is still challenging, due to the lack of a simple analytical formula to calculate the SLO attainment (a.k.a., percentage of requests that meet TTFT requirement), given that the workload has diverse input, output lengths, and irregular arrival patterns. Gauging the SLO via real-testbed profiling is time-prohibitive. We thus resort to building a simulator to estimate the SLO attainment, assuming prior knowledge of the workload's arrival process and input and output length distributions. Although short-term interval is impossible to predict, the workload pattern over longer timescales (e.g., hours or days) is often predictable [33, 55]. DistServe fits a distribution from the history request traces and resamples new traces from the distribution as the input workload to the simulator to compute the SLO attainment. Next, DistServe simply enumerates the placements and finds the maximum rate that meets the SLO attainment target with binary search and simulation trials.

Algorithm 1 outlines the process. We enumerate all feasible parallel configurations, subject to cluster capacity limit, for both prefill and decoding instances. Then, for a specific prefill phase configuration, we use simu_prefill to simulate

Algorithm 2 Low Node-Affinity Placement Algorithm

Input: LLM G , #node limit per-instance N , #GPU per-node M , GPU memory capacity C , workload W , traffic rate R .

Output: the placement $best_plm$.

```
config* ← 0
for inter_op ∈ {1, 2, ..., N} do
  P ← get_intra_node_configs(G, M, C, inter_op)
  for Pp ∈ P do
    for Pd ∈ P do
      if Pp.num_gpus + Pd.num_gpus ≤ M then
        config ← (inter_op, Pp, Pd)
         $\hat{G}_p, \hat{G}_d \leftarrow \text{parallel}(G, config)$ 
        config.goodput ← simulate( $\hat{G}_p, \hat{G}_d, W$ )
        if  $\frac{config^*.goodput}{config^*.num\_gpus} < \frac{config.goodput}{config.num\_gpus}$  then
          config* ← config
n ←  $\lceil \frac{R}{config^*.goodput} \rceil$ 
best_plm ← (n, config*)
return best_plm
```

and find its maximum goodput via binary search (similarly for using simu_decode for decoding). After determining the optimal parallel configurations for both prefill and decoding instances, we replicate them to achieve the user-required overall traffic rate according to their goodput.

The complexity of Algorithm 1 is $O(NM^2)$, with N as the node limit per instance and M representing the typical number of GPUs per node in modern clusters (e.g., 8). The search space is manageable and the solving time is under 1.3 minutes in our largest setting, as demonstrated in §6.5.

Simulator building. Algorithm 1 relies on a simulator to estimate the goodput under various SLOs and SLO attainment goals given the workload and the parallelism plan. To build an accurate simulator, we analyze the FLOPs and the number of memory accesses for prefill and decoding phases respectively, and use a latency model to approximate the inference execution time. See details in Appendix A. The simulator aligns well with real profiling results, thanks to the high predictability of DNN workloads [23, 33], verified in §6.4.

By far, we have developed Algorithm 1 assuming we can place the prefill and decoding instance between any two nodes (or on the same node) of the cluster, and the KV cache transmission utilizes high bandwidth network. In many real clusters, GPUs inside a node access to high-bandwidth NVLINK while GPUs distributed across nodes have limited bandwidth. We next develop an algorithm to address this constraint.

4.2 Placement for Low Node-Affinity Cluster

A straightforward solution is to always colocate prefill and decoding instances on the same node, utilizing the NVLINK, which is commonly available inside a GPU node. For large models, e.g. with 175B parameters (350GB), we may be unable to even host a single pair of prefill and decoding instances

算法1 高节点亲和性放置算法

输入: 大语言模型 G , 每实例节点限制 N , 每节点GPU数量

M , GPU内存容量 C , 工作负载 W , 流量速率 R 。

输出: 最佳放置方案 $best_plm$ 。

```
配置p, 配置d ← 0, 0
对于 intra_op ∈ {1, 2, ..., M} 执行
  inter_op ∈ {1, 2, ...,  $\frac{N \times M}{intra\_op}$ } do
    if  $\frac{G.size}{inter\_op \times intra\_op} < C$  然后
      配置 ← (inter_op, intra_op)
       $\hat{G} \leftarrow \text{并行}(G, \text{配置})$ 
      配置.有效吞吐量 ← simu_预填充( $\hat{G}, W$ )
      如果  $\frac{config_p.有效吞吐量}{config_p.num\_GPU} < \frac{配置.有效吞吐量}{配置.num\_GPU}$  则
        配置p ← 配置
      配置.有效吞吐量 ← simu_decode( $\hat{G}, W$ ) 如果 configd.有效吞吐量  $\frac{configd.num\_GPU}{配置.有效吞吐量} < \frac{配置.有效吞吐量}{配置.num\_GPU}$  则
        配置d ← 配置
n, m ←  $\lceil \frac{R}{Rconfig_p.goodput} \rceil, \lceil \frac{R}{Rconfig_d.goodput} \rceil$ 
best_plm ← (n, configp, m, configd)
return best_plm
```

4.1 面向高节点亲和性集群的放置方案

在配备InfiniBand的高节点亲和性集群上, 跨节点的KV缓存传输开销可忽略不计。DistServe可以将预填充和解码实例部署到任意两个节点, 且不受约束。我们针对此类场景提出了一种两级放置算法: 首先, 分别优化预填充和解码实例的并行配置, 以实现阶段级最优每GPU良率; 然后, 使用复制来匹配整体流量速率。

然而, 为单一实例类型(例如预填充实例)找到最优并行配置仍然具有挑战性, 因为缺乏简单的解析公式来计算 SLO 达成率(也称为满足 TTFT 要求的请求比例), 而工作负载具有多样化的输入和输出长度以及不规则的到达模式。通过真实测试床进行性能分析以衡量 SLO 在时间上代价过高。因此, 我们转而构建一个模拟器来估计 SLO 达成率, 假设已知工作负载的到达过程以及输入和输出长度分布。虽然短期内不可能进行预测, 但更长时间尺度(例如数小时或数天)上的工作负载模式通常是可预测的 [33, 55]。DistServe 从历史请求轨迹中拟合一个分布, 并从该分布中重新采样新的轨迹作为模拟器的输入工作负载, 以计算 SLO 达成率。接下来, DistServe 简单地枚举放置方案, 并通过二分查找和仿真试验找到满足 SLO 达标目标的最大速率。

算法1概述了该过程。我们枚举所有受集群容量限制的、适用于预填充和解码实例的可行并行配置。然后, 对于特定的预填充阶段配置, 我们使用simu_prefill进行模拟

算法 2 低节点亲和性放置算法

输入: 大语言模型 G , 每实例节点限制 N , 每节点GPU数量

M , GPU内存容量 C , 工作负载 W , 流量速率 R 。

输出: 最优放置方案 plm 。

```
配置* ← 0
对于_的 ∈ {1, 2, ..., N}, 执行
  P ← get_节点内_配置_(G, M, C, 节点间_操作)
  对于 Pp ∈ P 执行
    对于 Pd ∈ P 执行
      如果 Pp.num_gpus + Pd.num_gpus ≤ M 则
        配置 ← (节点间_操作, Pp, Pd)
         $\hat{G}_{p_d} \leftarrow \text{并行}(G, \text{配置})$ 
        配置.goodput ← simulate( $\hat{G}_{p_d}, \hat{G}_d, W$ )
        if 配置.有效吞吐量  $\frac{配置.num\_GPU}{配置.有效吞吐量} < \frac{配置.有效吞吐量}{配置.num\_GPU}$  then 配置* ← 配置
n ←  $\lceil \frac{R}{R配置^*.有效吞吐量} \rceil$ 
最优_plm ← (n, 配置*)
return best_plm
```

并通过二分查找找到其最大有效吞吐量(使用simu_decode进行解码的模拟方法类似)。在确定预填充和解码实例的最优并行配置后, 我们对其进行复制, 以根据它们的有效吞吐量满足用户所需的整体流量速率。

算法1的复杂度为 $O(NM^2)$, 其中 N 是每个实例的节点限制, M 代表现代集群中每个节点的典型GPU数量(例如8)。搜索空间是可控的, 且在我们的最大设置中, 求解时间少于1.3分钟, 如§6.5所示。

模拟器构建。算法1依赖一个模拟器, 在给定工作负载和并行策略计划的情况下, 估计在不同服务等级目标和SLO达成目标下的有效吞吐量。为了构建一个精确的模拟器, 我们分别分析了预填充阶段和解码阶段的FLOPs和内存访问数量, 并使用延迟模型来近似推理执行时间。详见附录A。得益于第6.4节中验证的DNN工作负载的高可预测性, 模拟器的结果与真实的性能分析结果高度一致 [23, 33]。

迄今为止, 我们在开发算法1时, 假设可以在集群的任意两个节点之间(或同一节点上)放置预填充实例和解码实例, 并且KV缓存传输利用高带宽网络。在许多实际集群中, 节点内部的GPU通过高带宽NVLink进行访问, 而分布在不同节点上的GPU则受限于较低的带宽。接下来, 我们将开发一种算法来解决这一约束。

4.2 低节点亲和性集群的放置方案

一种直接的解决方案是始终将预填充和解码实例共置在同一节点上, 并利用节点内部通常配备的NVLink。对于大模型, 例如具有1750亿参数(350GB)的模型, 我们甚至可能无法

in an 8-GPU node ($80G \times 8 = 640G < 350 \times 2GB$). We incorporate this as additional placement constraints and co-optimize it with model parallelism, presented in Algorithm 2.

The key insight is that KV cache transfer occurs exclusively between corresponding layers of prefill and decoding instances. Leveraging inter-op parallelism, we group layers into stages and divide each instance into segments, termed as *instance segments*, with each segment maintaining one specific inter-op stage. By colocating prefill and decoding segments of the same stage within a single node, we force the transfer of intermediate states to occur only via NVLINK. Inside a node, we set the same parallelism and resource allocation for segments of the same instance. Given the typical limitation of GPUs per node (usually 8), we can enumerate possible configurations inside one node and use the simulator to identify the configurations that yield the best goodput.

As outlined in Algorithm 2, we begin by enumerating inter-op parallelism degrees to get all the possible instance segments. For each segment, we get all possible intra-node parallelism configurations by calling `get_intra_node_configs`. Then we use simulation to find the optimal one and replicate it to satisfy the target traffic rate.

4.3 Online scheduling

The runtime architecture of DistServe is shown in Figure 6. DistServe operates with a simple FCFS scheduling policy. All incoming requests arrive at a centralized controller, then dispatched to the prefill instance with the shortest queue for prefill processing, followed by dispatch to the least loaded decoding instance for decoding steps. This setup, while simple, is optimized with several key enhancements tailored to the nuances of real-world workloads.

Reducing pipeline bubbles. To mitigate the pipeline bubbles caused by non-uniform prompt lengths (§3.3), we schedule the requests in a way that balances the execution time across all batches in the pipeline. This is achieved by noting that, for both prefill and decoding instances, the number of new tokens in the batch is a reliable indicator of the batch’s real execution time. For prefill instances, we profile the target model and GPU to figure out the shortest prompt length L_m needed to saturate the GPU. We schedule prefill batches with a total sequence length close to L_m , by either batching multiple requests shorter than L_m or individually scheduling requests longer than L_m . For decoding instances, we set L_m as the largest batch size.

Combat busrtiness. Burstiness in workloads can cause a deluge of KV caches to transfer from prefill to decoding instances, risking memory overload on decoding instances. To circumvent this, DistServe employs a “pull” method for KV cache transmission rather than a “push” approach – decoding instances fetch KV cache from prefill instances *as needed*, using the GPU memory of prefill instances as a queuing buffer. This way, the prefill instance can continue handling other

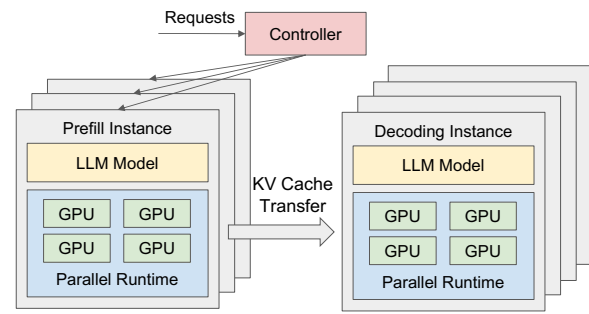


Figure 6: DistServe Runtime System Architecture

prefill jobs by simply retaining the KV Cache in the GPU memory after processing the prompt. Hence, each type of instance operates at its own pace without complex coordination.

Replanning. The resource and parallelism plan in DistServe is optimized for a specific workload pattern, which may become suboptimal if the workload pattern changes over time. DistServe implement periodic replanning. A workload profiler monitors key parameters such as the average input and output length of the requests, the average arrival rate, etc. If a significant pattern shift is detected, DistServe will trigger a rerun of the placement algorithm based on recent historical data. This process is expedient – the proposed algorithm runs in seconds (§6.5) and reloading LLM weights can be completed within minutes – far shorter than the hourly scale at which real-world workload variations tend to occur.

Preemption and fault tolerance. DistServe does not implement advanced runtime policies like preemption [26] and fault tolerance [58], which are complementary to disaggregation. Nevertheless, we discuss how they fit into DistServe. In DistServe, the FCFS policy can lead to a “convoy effect”, where longer requests block shorter ones in the prefill stage. Incorporating preemptive strategies, as suggested in existing literature [53], could enhance efficiency and is feasible within our system’s architecture. While not a primary focus in the current DistServe, fault tolerance is a critical aspect for consideration. In traditional colocation- and replication-based systems, a fault in one instance typically does not disrupt other replica instances. However, in DistServe, the dependency between prefill and decoding instances introduces the risk of fault propagation. For example, a fault in a single decoding instance mapped to multiple prefill instances could potentially cripple the entire service and cluster. We leave both as future work.

5 Implementation

DistServe is an end-to-end distributed serving system for LLMs with a placement algorithm module, a RESTful API frontend, an orchestration layer, and a parallel execution engine. The algorithm module, frontend, and orchestration layer are implemented with 6.5K lines of Python code. The parallel execution engine is implemented with 8.1K lines of

在8-GPU节点 ($80G \times 8 = 640G < 350 \times 2GB$) 中托管一对预填充和解码实例。我们将此作为额外的放置约束，并与模型并行进行协同优化，如算法 2 所示。

关键洞察在于，KV缓存的传输仅发生在预填充实例与解码实例的对应层之间。利用算子间并行，我们将层分组为阶段，并将每个实例划分为实例分段，每个实例分段维护一个特定的算子间阶段。通过将相同阶段的预填充分段和解码分段置于同一节点内部共置，我们强制中间状态的传输仅通过NVLink进行。在节点内部，我们为同一实例的实例分段设置相同的并行性和资源配置。考虑到每节点GPU数量的典型限制（通常为8个），我们可以枚举单节点内的可能配置，并使用模拟器来识别能产生最佳有效吞吐量的配置。

如算法 2 所述，我们首先枚举算子间并行度，以获得所有可能的实例分段。对于每个分段，我们通过调用 `get_intra_node_configs` 获取所有可能的节点内并行配置。然后，我们使用仿真来找到最优配置，并将其复制以满足目标流量速率。

4.3 在线调度

DistServe 的运行时架构如图 6 所示。DistServe 采用一种简单的先来先服务调度策略运行。所有传入请求首先到达集中式控制器，随后被分派到队列最短的预填充实例进行预填充处理，接着分派到负载最轻的解码实例执行解码步骤。这种配置虽然简单，但针对真实世界工作负载的细节进行了优化，并包含多项关键增强功能。

减少流水线气泡。为缓解由非统一的提示词长度 (§3.3) 引起的流水线气泡，我们以一种平衡流水线中所有批处理执行时间的方式调度请求。这是通过以下方式实现的：注意到对于预填充实例和解码实例，批处理中新令牌的数目是该批处理实际执行时间的可靠指标。对于预填充实例，我们对目标模型和 GPU 进行配置分析，以确定使 GPU 达到饱和和所需的最短提示长度 L_m 。我们通过批处理多个短于 L_m 的请求或单独调度长于 L_m 的请求，来调度总序列长度接近 L_m 的预填充实例的批处理。对于解码实例，我们将 L_m 设置为最大批大小。

应对突发性。工作负载中的突发性会导致大量KV缓存从预填充实例传输到解码实例，从而使解码实例面临内存过载的风险。为规避这一问题，DistServe采用“拉取”方式进行KV缓存传输，而非“推送”方式——解码实例在需要时从预填充实例拉取KV缓存，并利用预填充实例的GPU内存作为排队缓冲区。这样一来，预填充实例可以在处理完提示词后，只需将KV缓存保留在GPU内存中，即可继续处理其他

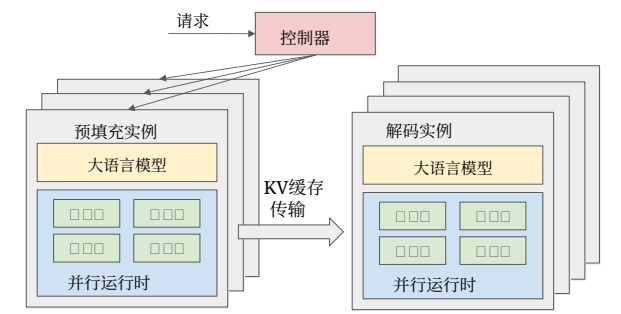


图6: DistServe运行时系统架构

预填充任务。因此，每种类型的实例都能以自身的节奏运行，而无需复杂的协调。

重新规划。DistServe 中的资源与并行策略计划是针对特定的工作负载模式进行优化的，如果工作负载模式随时间发生变化，该计划可能会变得不再最优。DistServe 实现了周期性重新规划。工作负载分析器会监控关键参数，例如请求的平均输入和输出长度、平均到达率等。如果检测到显著的模式变化，DistServe 将基于近期的历史数据触发放置算法的重新运行。这一过程十分快捷——所提出的算法在数秒内即可运行完毕（见第 6.5 节），且重新加载大语言模型权重可在几分钟内完成——这远短于实际工作负载波动通常发生的小时级时间尺度。

抢占与容错。DistServe 并未实现诸如抢占 [26] 和容错 [58], 等高级运行时策略，这些策略与解聚合是互补的。尽管如此，我们仍探讨了这些策略如何融入 DistServe。在 DistServe 中，先到先服务策略可能导致“护航效应”，即较长的请求在预填充阶段阻塞较短的请求。正如现有文献 [53], 所建议的那样，引入抢占策略可以提高效率，并且在我们系统的架构中是可行的。虽然容错不是当前 DistServe 的主要关注点，但它是需要考虑的一个关键方面。在传统的基于共置和复制的系统中，单个实例的故障通常不会干扰其他副本实例。然而，在 DistServe 中，预填充实例和解码实例之间的依赖关系引入了故障传播的风险。例如，映射到多个预填充实例的单个解码实例的故障，可能会使整个服务和集群瘫痪。我们将这两者留作未来工作。

5 实现

DistServe 是一个面向大语言模型的端到端分布式服务系统，包含放置算法模块、RESTful API 前端、编排层以及并行执行引擎。算法模块、前端和编排层使用 6.5K 行 Python 代码实现。并行执行引擎使用 8.1K 行

Application	Model Size	TTFT	TPOT	Dataset
Chatbot OPT-13B	26GB	0.25s	0.1s	ShareGPT [8]
Chatbot OPT-66B	132GB	2.5s	0.15s	ShareGPT [8]
Chatbot OPT-175B	350GB	4.0s	0.2s	ShareGPT [8]
Code Completion OPT-66B	132GB	0.125s	0.2s	HumanEval [14]
Summarization OPT-66B	132GB	15s	0.15s	LongBench [13]

Table 1: Workloads in evaluation and latency requirements.

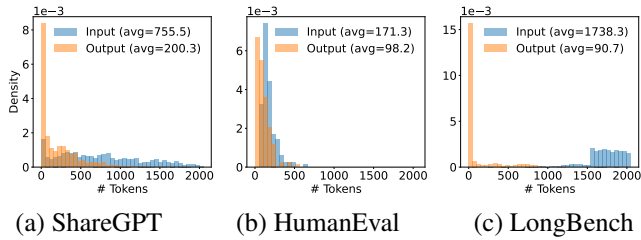


Figure 7: The input and output length distributions of (a) ShareGPT, (b) HumanEval, and (c) LongBench datasets.

C++/CUDA code.

The placement algorithm module implements the algorithm and the simulator mentioned in §4 which gives the placement decision for a specific model and cluster setting. The frontend supports an OpenAI API-compatible interface where clients can specify the sampling parameters like maximum output length and temperature. The orchestration layer manages the prefill and decoding instances, responsible for request dispatching, KV cache transmission, and results delivery. It utilizes NCCL [6] for cross-node GPU communication and asynchronous CudaMemcpy for intra-node communication, which avoids blocking the GPU computation during transmission. Each instance is powered by a parallel execution engine, which uses Ray [35] actor to implement GPU workers that execute the LLM inference and manage the KV Cache in a distributed manner. It integrates many recent LLM optimizations like continuous batching [54], FlashAttention [20], PagedAttention [32] and supports popular open-source LLMs such as OPT [56] and LLaMA [51].

6 Evaluation

In this section, we evaluate DistServe under different sizes of LLMs ranging from 13B to 175B and various application datasets including chatbot, code-completion, and summarization. The evaluation shows that DistServe consistently outperforms the current state-of-the-art system across all the settings (§6.2). Specifically, DistServe can handle up to $7.4\times$ higher rates and $12.6\times$ more stringent SLO while meeting the latency requirements for over 90% requests. Additionally, we analyze the latency breakdown in DistServe to show the communication overhead is insubstantial thanks to our bandwidth-aware placement algorithm (§6.3) and do ablation studies of our techniques (§6.4). Finally, we profile the execution time of our placement algorithm (§6.5).

6.1 Experiments Setup

Cluster testbed. We deploy DistServe on a cluster with 4 nodes and 32 GPUs. Each node has 8 NVIDIA SXM A100-80GB GPUs connected with NVLINK. The cross-node bandwidth is 25Gbps. Due to the limited cross-node bandwidth, we use the low node-affinity placement algorithm (§2) for DistServe in most of the experiments except for the ablation study (§6.4) which uses simulation.

Model and workloads setup. Similar to prior work on LLM serving [32], we choose the OPT [56] model series, which is a representative LLM family widely used in academia and industry. Newer GPT model families are adopting memory-efficient attention mechanisms like GQA [10] and MQA [44]. DistServe will show better performance on these models because the transmission overhead is lower due to the decrease in KV cache size. We choose OPT which uses the classic MHA [52] to put enough pressure on the transmission overhead. We use FP16 precision in all experiments. For workloads, as shown in Table 1, We choose three typical LLM applications and set the SLOs empirically based on their service target because there exists no available SLO settings for these applications as far as we know. For each application, we select a suitable dataset and sample requests from it for evaluation. Since all the datasets do not include timestamps, we generate request arrival times using Poisson distribution with different request rates. Due to the space limit, we test the chatbot workload on all three OPT models and the other two workloads on OPT-66B, which matches the largest size in the recent open-source LLM series [51].

- **Chatbot** [1]: We use the ShareGPT dataset [8] for the chatbot application, which is a collection of user-shared conversations with ChatGPT. For OPT-13B, the TTFT SLO is set to 0.25s for responsiveness and the TPOT SLO is set to 0.1s which is higher than the normal human read speed. For OPT-66B and OPT-175B, we slightly relax the two SLOs due to the increase in model execution latency.
- **Code completion** [14]: We use the HumanEval [14] dataset for the code completion task. It includes 164 programming problems with a function signature or docstring which is used to evaluate the performance of code completion models. Since the code completion model is used as a personal real-time coding assistant, we set both SLOs to be stringent.
- **Summarization** [5]: It is a popular LLM task to generate a concise summary for a long article, essay, or even an academic paper. We use LongBench [13] dataset which contains the summarization task⁴. As shown in Figure 7, LongBench has much longer input lengths than the other two datasets. So we set a loose TTFT SLO but require a stringent TPOT.

Metrics. We use *SLO attainment* as the major evaluation metric. Under a specific SLO attainment goal (say, 90%), we are

⁴We capped the input lengths in LongBench because OPT’s absolute positional embedding only supports a maximum length of 2048.

应用	模型大小	TTFT	TPOT	数据集
聊天机器人 OPT-13B	26GB	0.25s	0.1s	ShareGPT [8]
聊天机器人 OPT-66B	132GB	2.5s	0.15s	ShareGPT [8]
聊天机器人 OPT-175B	350GB	4.0s	0.2s	ShareGPT [8]
代码补全 OPT-66B	132GB	0.125s	0.2s	HumanEval [14]
摘要生成 OPT-66B	132GB	15s	0.15s	LongBench [13]

表 1: 评估中的工作负载及延迟要求。

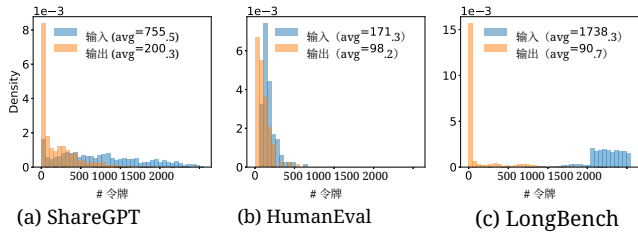


图 7: (a) ShareGPT、(b) HumanEval 和 (c) LongBench 数据集的输入和输出长度分布。

C++/CUDA 代码。

放置算法模块实现了第4节中提到的算法和模拟器，针对特定的模型和集群配置给出放置决策。前端支持兼容OpenAI API的接口，客户端可以指定采样参数，例如最大输出长度和温度。编排层管理预填充实例和解码实例，负责请求分发、KV缓存传输和结果传输。它利用NCCL [6] 进行跨节点GPU通信，并采用异步CudaMemcpy进行节点内通信，从而在传输过程中避免阻塞GPU计算。每个实例由并行执行引擎提供支持，该引擎使用Ray [35] actor来实现GPU工作进程，以分布式方式执行大语言模型推理并管理KV缓存。它集成了多种最新的大语言模型优化技术，例如连续批处理 [54]、FlashAttention [20]和分页注意力 [32]，并支持OPT [56] 和LLaMA [51]等流行的开源大语言模型。

6 评估

在本节中，我们评估了 DistServe 在不同规模（从 13B 到 175B）的大语言模型以及多种应用数据集上的表现，这些数据集包括聊天机器人、代码补全和摘要。评估结果显示，在所有设置中 (§6.2)，DistServe 始终优于当前最先进的系统。具体而言，DistServe 能够在满足超过 90% 请求的延迟要求的同时，处理高达 $7.4\times$ 倍的速率，并支持 $12.6\times$ 更严格的 SLO。此外，我们分析了 DistServe 中的延迟分解，以证明得益于我们的带宽感知放置算法 (§6.3)，通信开销微乎其微，并对我们的技术进行了消融研究 (§6.4)。最后，我们对放置算法的执行时间进行了性能分析 (§6.5)。

6.1 实验设置

集群测试床。我们部署了 DistServe，在一个包含 4 个节点和 32 个 GPU 的集群上运行。每个节点包含 8 个通过 NVLink 连接的英伟达 SXM A100-80GB GPU。跨节点带宽为 25Gbps。由于跨节点带宽有限，我们在大多数实验中使用低节点亲和性放置算法 (§2) 来部署 DistServe，除了使用仿真的消融研究 (§6.4) 之外。

模型和工作负载设置。与先前的 LLM 服务工作 [32]类似，我们选择了 OPT [56] 模型系列，这是一个在学术界和工业界广泛使用的代表性 LLM 家族。较新的 GPT 模型家族正在采用 GQA [10] 和 MQA [44]等内存高效的注意力机制。DistServe 在这些模型上将表现出更好的性能，因为 KV缓存大小的减少使得传输开销降低。我们选择使用经典 MHA [52] 的 OPT，以施加足够的传输开销压力。我们在所有实验中使用 FP16 精度。对于工作负载，如表 1 所示，我们选择了三个典型的大语言模型应用，并根据其服务目标经验性地设置服务等级目标，因为据我们所知，这些应用没有可用的 SLO 设置。对于每个应用，我们选择一个合适的数据集，并从中采样请求进行评估。由于所有数据集都不包含时间戳，我们使用不同请求速率的泊松分布生成请求到达时间。受空间限制，我们在所有三个 OPT 模型上测试聊天机器人工作负载，并在 OPT-66B 上测试其他两个工作负载，这与最近开源 LLM 系列 [51]中的最大尺寸相匹配。

- 聊天机器人 [1]: 我们使用ShareGPT数据集 [8] 用于聊天机器人应用，该数据集是一个与ChatGPT的用户共享对话的集合。对于OPT-13B，为确保响应性，TTFT SLO设置为0.25秒，TPOT服务等级目标设置为0.1秒，这高于普通人类的阅读速度。对于OPT-66B和OPT-175B，由于模型执行延迟的增加，我们略微放宽了这两项服务等级目标。
- 代码补全 [14]: 我们使用HumanEval [14] 数据集用于代码补全任务。它包含164个带有函数签名或文档字符串的编程问题，用于评估代码补全模型的性能。由于代码补全模型被用作个人实时编程助手，我们将两项服务等级目标都设置得较为严格。
- 摘要生成 [5]: 这是一个流行的大语言模型任务，旨在为长文章、论文甚至学术论文生成简洁的摘要。我们使用包含摘要生成任务的LongBench [13] 数据集⁴。如图7所示，LongBench的输入长度比其他两个数据集长得多。因此，我们设置了一个宽松的TTFT SLO，但要求严格的TPOT。

指标。我们使用SLO达成率作为主要评估指标。在特定的SLO达成目标下（例如90%），我们

⁴我们将 LongBench 中的输入长度进行了限制，因为 OPT 的绝对位置嵌入仅支持最大长度为 2048。

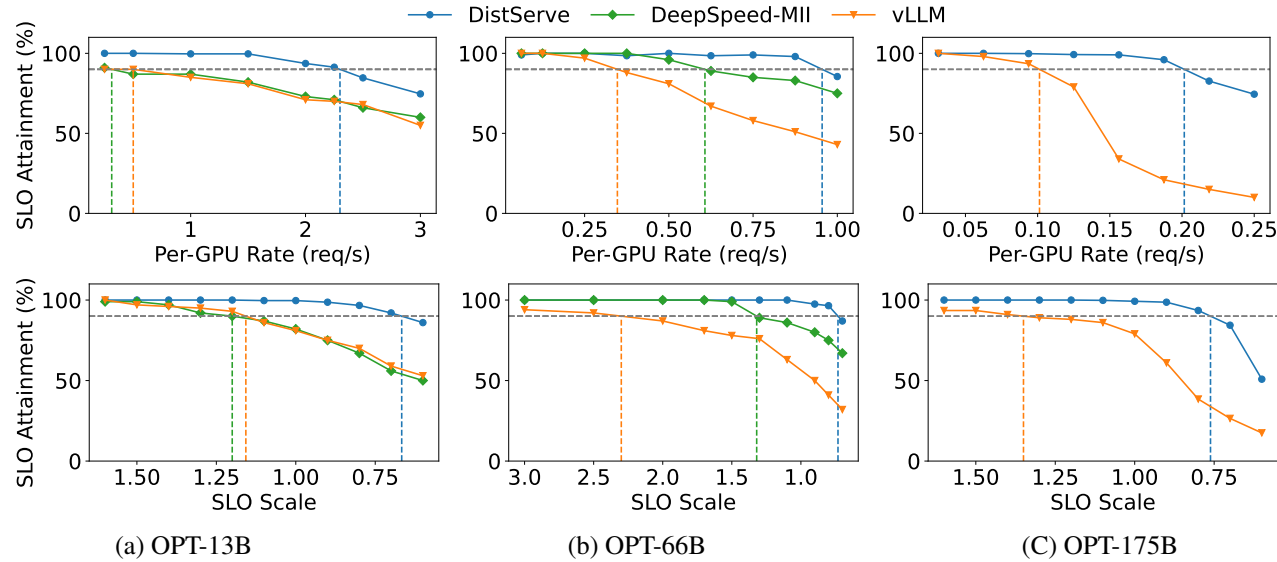


Figure 8: Chatbot application with OPT models on the ShareGPT dataset.

concerned with two things: the maximum per-GPU goodput and the minimal SLO the system can handle. We are particularly interested in an SLO attainment of 90% (indicated by the vertical lines in all curve plots), but will also vary the rate and latency requirements to observe how the SLO attainment changes. We also include the results in the Appendix for an SLO attainment of 99% to show the system performance under a more stringent SLO attainment target.

Baselines. We compare DistServe to two baseline systems:

- **vLLM [32]:** vLLM is a representative LLM serving system widely used in both academia and industry. It supports *continuous batching* [54] to increase throughput and *paged-attention* [32] to reduce memory fragmentation during KV cache allocation. However, it colocates the prefill and decoding computation to maximize the overall system throughput and struggles to meet the latency requirements cost-efficiently. Since vLLM only supports intra-op parallelism, we follow previous work [32] to set intra-op equals 1, 4, and 8 for the three OPT models, respectively.
- **DeepSpeed-MII [3]:** DeepSpeed Model Implementations for Inference (MII) supports *chunked-prefill* by decomposing long prompts into smaller chunks and composing with short prompts to exactly fill a target token budget. It mitigates but cannot eliminate the prefill-decoding interference caused by the long prefill job. We set its intra-op the same as vLLM for OPT-13B and OPT-66B for a fair comparison. However, DeepSpeed-MII cannot serve OPT-175B whose *vocab_size* = 50272 because its underlying kernel implementation requires *vocab_size/intra_op* is a multiple of 8 where intra-op equals 8 does not satisfy. Setting intra-op equals 4 can satisfy this requirement but will cause the out-of-memory issue.

6.2 End-to-end Experiments

In this Section, we compare the end-to-end performance of DistServe against the baselines on real application datasets.

Chatbot. We evaluate the performance of DistServe on the chatbot application for all three OPT models. The first row of Figure 8 illustrates that when we gradually increase the rate, more requests will violate the latency requirements and the SLO attainment decreases. The vertical line shows the maximum per-GPU rate the system can handle to meet latency requirements for over 90% of the requests.

On the ShareGPT dataset, DistServe can sustain $2.0\times$ – $4.6\times$ higher request rate compared to vLLM. This is because DistLLM eliminates the prefill-decoding interference through disaggregation. Two phases can optimize their own objectives by allocating different resources and employing tailored parallelism strategies. Specifically, by analyzing the chosen placement strategy⁵ for 175B, we find the prefill instance has inter-op = 3, intra-op = 3; and the decoding instance has inter-op = 3, intra-op = 4. Under this placement, DistServe can effectively balance the load between the two instances on ShareGPT, meeting latency requirements at the lowest cost. This non-trivial placement strategy is challenging to manually find, proving the effectiveness of the algorithm. In the case of vLLM, collocating prefill and decoding greatly slows down the decoding phase, thereby significantly increasing TPOT. Due to the stringent TPOT requirements of chatbot applications, although vLLM meets the TTFT SLO for most requests, the overall SLO attainment is dragged down by a large number of requests that violate the TPOT SLO. Compared to DeepSpeed-MII, DistServe can sustain $1.6\times$ – $7.4\times$ higher request rate. DeepSpeed-MII shows better performance

⁵All the placements chosen by DistServe can be found in Appendix B.

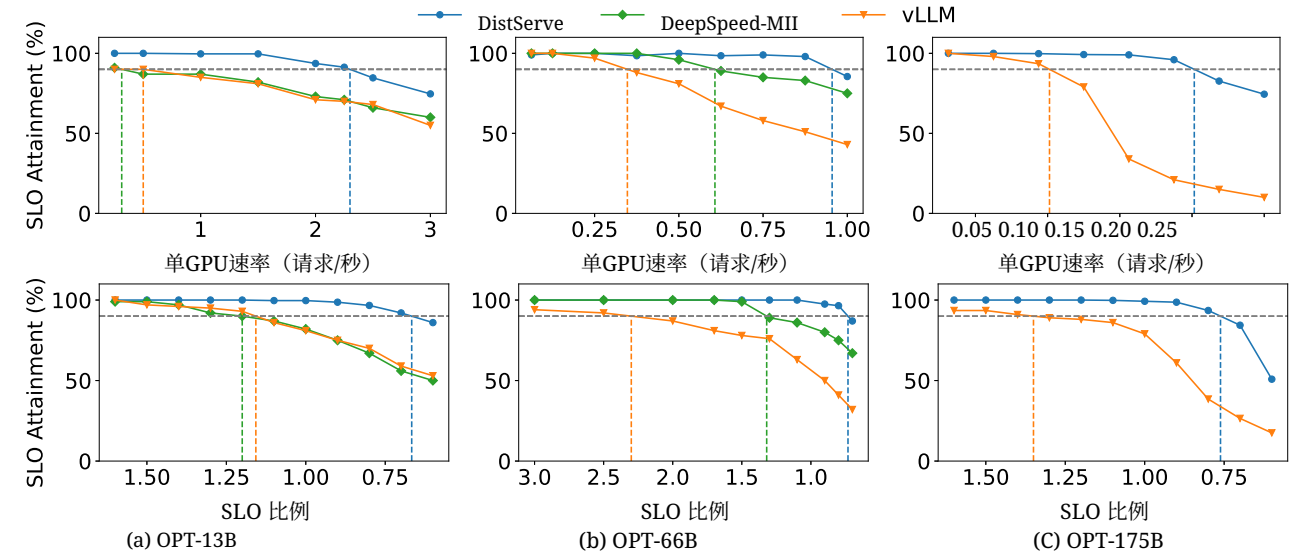


图8：基于OPT模型的聊天机器人应用在ShareGPT数据集上的表现。

关注两件事：每GPU有效吞吐量的最大值，以及系统能够处理的最小SLO。我们尤其关注90%的SLO达成率（在所有曲线图中用竖线表示），但同时也会改变速率和延迟要求，以观察SLO达成率的变化。我们还在附录中提供了99% SLO达成率的结果，以展示系统在更严格的SLO达标目标下的性能。

基线系统。我们将 DistServe 与两个基线系统进行比较：

- **vLLM [32]:** vLLM 是一个代表性的大语言模型服务系统，在学术界和工业界均被广泛采用。它支持连续批处理 [54] 以提升吞吐量，并支持分页注意力机制 [32] 以减少 KV 缓存分配期间的内存碎片。然而，vLLM 采用预填充和解码计算共置的策略以最大化整体系统吞吐量，但在高效满足延迟要求方面面临挑战。由于 vLLM 仅支持算子内并行，我们参考先前的工作 [32]，分别为三个 OPT 模型将算子内并行度设置为 1、4 和 8。
- **DeepSpeed-MII [3]:** 用于推理的DeepSpeed模型实现 (MII) 通过将长提示词分解为较小的分块，并与短提示词组合以恰好填满目标令牌预算，从而支持分块预填充。它减轻了但无法消除由长预填充任务引起的预填充-解码干扰。为了进行公平比较，我们将针对OPT-13B和OPT-66B的算子内并行设置为与vLLM相同。然而，DeepSpeed-MII无法服务OPT-175B，因为其底层内核实现要求词表大小 = 50272 满足词表大小/算子内并行是8的倍数这一条件，而将算子内并行设为8则不满足该条件。将算子内并行设为4可以满足此要求，但会导致内存溢出问题。

6.2 端到端实验

在本节中，我们在真实应用数据集上比较了DistServe与基线系统的端到端性能。

聊天机器人。我们评估了DistServe在三个OPT模型上的聊天机器人应用性能。图8的第一行表明，随着速率逐渐增加，将有更多请求违反延迟要求，且SLO达成率随之下降。垂直线显示了系统为满足超过90%的请求的延迟要求所能处理的最大每GPU速率。

在ShareGPT数据集上，DistServe能够维持比vLLM高 $2.0\times$ – $4.6\times$ 的请求速率。这是因为DistLLM通过解耦消除了预填充-解码干扰。两个阶段可以通过分配不同的资源和采用定制的并行策略来优化各自的目标。具体而言，通过分析175B的放置策略⁵，我们发现预填充实例具有算子间并行 = 3、算子内并行 = 3；而解码实例具有算子间并行 = 3、算子内并行 = 4。在此放置方案下，DistServe能够有效地在ShareGPT数据集上的两个实例之间平衡负载，以最低成本满足延迟要求。这种非平凡的放置策略很难通过人工找到，证明了该算法的有效性。对于vLLM，预填充和解码的共置会严重拖慢解码阶段，从而显著增加TPOT。由于聊天机器人应用对TPOT要求严格，尽管vLLM在大多数请求中满足了TTFT SLO，但整体SLO达成率因大量违反TPOT服务等级目标的请求而显著下降。与DeepSpeed-MII相比，DistServe能够维持高 $1.6\times$ – $7.4\times$ 的请求速率。DeepSpeed-MII在更大模型上表现出更好的性能

⁵All DistServe 选择的所有放置方案可在附录中找到 附录B

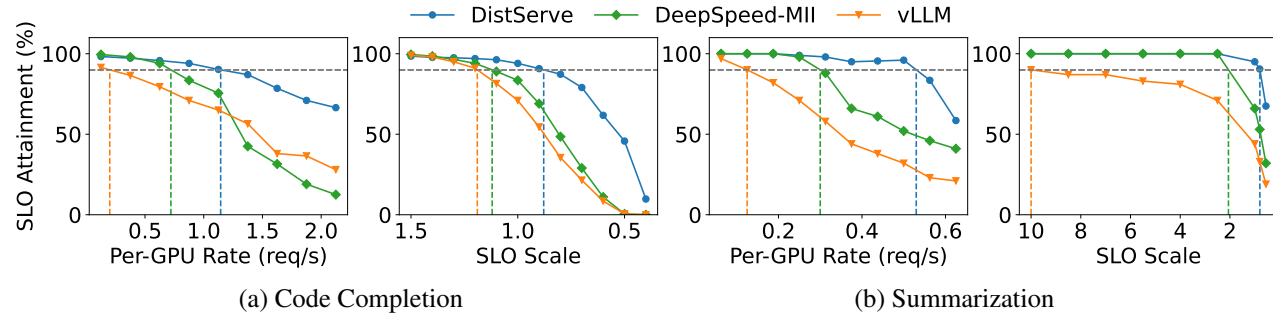


Figure 9: Code completion and summarization tasks with OPT-66B on HumanEval and LongBench datasets, respectively.

on larger models because the prefill job is larger and chunked-prefill mitigates the interference to some extent. However, due to the reasons discussed in §2.3, chunked prefill is slower than full prefill, so it struggles to meet the TTFT SLO as a sacrifice for better TPOT.

The second row of Figure 8 indicates the robustness to the changing latency requirements of the two systems. We fix the rate and then linearly scale the two latency requirements in Table 1 simultaneously using a parameter called *SLO Scale*. As SLO Scale decreases, the latency requirement is more stringent. We aim to observe the most stringent SLO Scale that the system can withstand while still achieving the attainment target. Figure 8 shows that DistServe can achieve $1.8\times-3.2\times$ more stringent SLO than vLLM and $1.7\times-1.8\times$ more stringent SLO than DeepSpeed-MII, thus providing more engaging service quality to the users.

Code completion. Figure 9(a) shows the performance of DistServe on the code completion task when serving OPT-66B. DistServe can sustain $5.7\times$ higher request rate and $1.4\times$ more stringent SLO than vLLM. Compared to DeepSpeed-MII, DistServe can sustain $1.6\times$ higher request rate and $1.4\times$ more stringent SLO. As a real-time coding assistant, the code completion task demands lower TTFT than chatbot, this leads to both systems ultimately being constrained by the TTFT requirement. However, in comparison, by eliminating the interference of the decoding jobs and automatically increasing intra-operation parallelism in prefill instances through the searching algorithm, DistServe reduces the average latency of the prefill jobs, thereby meeting the TTFT requirements of more requests.

Summarization. Figure 9(b) shows the performance of DistServe on the summarization task when serving OPT-66B. DistServe achieves $4.3\times$ higher request rate and $12.6\times$ more stringent SLO than vLLM. Compared to DeepSpeed-MII, DistServe achieves $1.8\times$ higher request rate and $2.6\times$ more stringent SLO. The requests sampled from LongBench dataset have long input lengths, which brings significant pressure to the prefill computation. However, due to the loose requirement of TTFT for the summarization task, the TPOT service quality becomes particularly important. Since vLLM collocates prefill and decoding phases, it experiences a greater

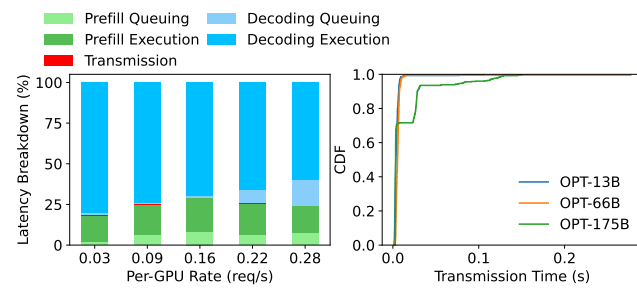


Figure 10: *Left:* Latency breakdown when serving OPT-175B on ShareGPT dataset with DistServe. *Right:* The CDF function of KV Cache transmission time for three OPT models.

slowdown in the decoding phase with long prefill jobs and fails to meet the TPOT requirement.

The results above are all under the 90% SLO attainment target. We observe that DistServe can have better performance under a more stringent attainment target (say, 99%) and present the results in Appendix C.

6.3 Latency Breakdown

To understand DistServe’s performance in detail, we make a latency breakdown of the requests in DistServe. We divide the processing lifecycle of a request in DistServe into five stages: *prefill queuing*, *prefill execution*, *transmission*, *decoding queuing*, and *decoding execution*. The total time consumed by all requests in each stage is then summed up to determine their respective proportions in the system’s total execution time.

Figure 10(a) shows the latency breakdown for the OPT-175B models on the ShareGPT dataset. We chose OPT-175B because the KV Cache transmission is more demanding for larger models. In fact, even for OPT-175B, the KV Cache transmission only accounts for less than 0.1% of the total latency. Even by examining the CDF of the absolute transmission time shown in Figure 10(b), we observe that over 95% of requests experience a delay of less than 30ms, despite our testbed having only limited cross-node bandwidth. This is due to the algorithm described in §4.2, where we require the prefill and decoding instance to maintain the same stage on one machine, enabling the use of intra-node NVLINK bandwidth for transmission, thus significantly reducing transmission delay.

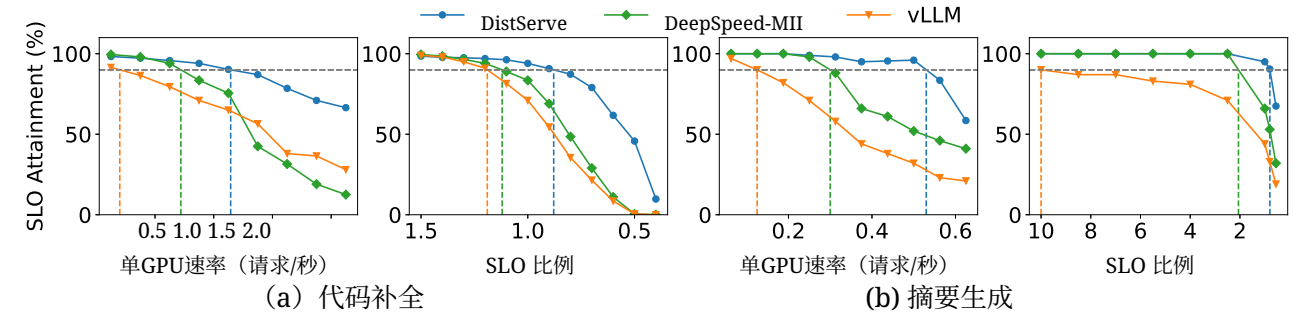


图9: 在 HumanEval 和 LongBench 数据集上, 分别使用 OPT-66B 进行的代码补全和摘要生成任务。

, 因为预填充任务更大, 且分块预填充在一定程度上减轻了干扰。然而, 如§2.3所述, 分块预填充比完整预填充更慢, 因此它难以在牺牲更好的TPOT的同时满足 TTFT SLO。

图8 的第二行展示了这两个系统对变化的延迟要求的鲁棒性。我们固定速率, 然后使用一个名为 SLO 比例的参数, 同时线性缩放表1中的两个延迟要求。随着 SLO 比例的降低, 延迟要求变得更加严格。我们旨在观察系统在仍能达成目标的前提下所能承受的最严格的 SLO 比例。图8 显示, DistServe 能够实现比 vLLM 更严格的 $1.8\times-3.2\times$ SLO, 以及比 DeepSpeed-MII 更严格的 $1.7\times-1.8\times$ SLO, 从而为用户提供更具吸引力的服务质量。

代码补全。图9(a) 展示了 DistServe 在为 OPT-66B 提供服务时, 在代码补全任务上的表现。与 vLLM 相比, DistServe 能够维持更高的请求速率 (高 $5.7\times$) 和更严格的 SLO (严格 $1.4\times$)。与 DeepSpeed-MII 相比, DistServe 能够维持更高的请求速率 (高 $1.6\times$) 和更严格的 SLO (严格 $1.4\times$)。作为实时编码助手, 代码补全任务对 TTFT 的要求比聊天机器人更低, 这导致两个系统最终都受到 TTFT 要求的限制。然而, 相比之下, DistServe 通过消除解码任务的干扰, 并利用搜索算法自动增加预填充实例中的操作并行度, 降低了预填充任务的平均延迟, 从而满足了更多请求的 TTFT 要求。

摘要生成。图9(b)展示了在提供 OPT-66B 服务时, DistServe 在摘要生成任务上的性能。与 vLLM 相比, DistServe 实现了 $4.3\times$ 更高的请求速率和 $12.6\times$ 更严格的 SLO。与 DeepSpeed-MII 相比, DistServe 实现了 $1.8\times$ 更高的请求速率和 $2.6\times$ 更严格的 SLO。从 LongBench 数据集中采样的请求具有较长的输入长度, 这给预填充计算带来了显著的压力。然而, 由于摘要生成任务对 TTFT 的要求较为宽松, TPOT 的服务质量变得尤为重要。由于 vLLM 将预填充阶段和解码阶段合并, 它在处理长预填充任务时会在解码阶段出现更严重的

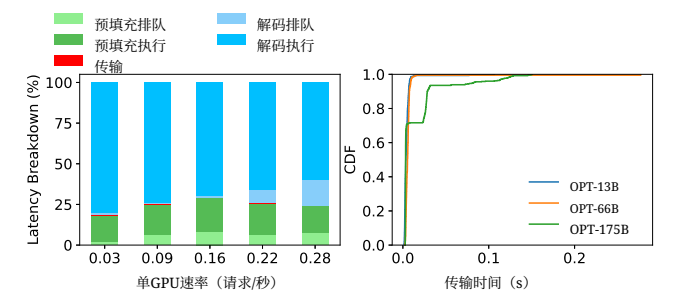


图10: 左图: 在ShareGPT数据集上使用DistServe提供 OPT-175B服务时的延迟分解。右图: 三个OPT模型的KV缓存传输时间的CDF函数。

减速, 并无法满足 TPOT 要求。

上述结果均基于90%的SLO达标目标。我们观察到, DistServe在更严格的目标 (例如99%) 下可以取得更好的性能, 并将结果列于附录C。

6.3 延迟分解

为了详细了解DistServe的性能, 我们对DistServe中的请求进行了延迟分解。我们将DistServe中一个请求的处理生命周期划分为五个阶段: 预填充排队、预填充执行、传输、解码排队以及解码执行。随后, 汇总每个阶段中所有请求所消耗的总时间, 以确定各阶段在系统总执行时间中的占比。

图10(a)展示了OPT-175B模型在ShareGPT数据集上的延迟分解。我们选择OPT-175B是因为对于更大的模型, KV缓存传输的需求更为苛刻。事实上, 即使是对于 OPT-175B, KV缓存传输也仅占总延迟的不到0.1%。通过观察图10(b)所示的绝对传输时间的CDF, 我们可以发现, 尽管我们的测试床仅具备有限的跨节点带宽, 但超过95%的请求所经历的延迟仍小于30毫秒。这是由于§4.2中描述的算法所致, 该算法要求预填充实例和解码实例在同一台机器上保持相同的阶段, 从而能够利用节点内NVLink带宽进行传输, 显著降低了传输延迟。

Rate (req/s)	vLLM		DistServe-Low	
	Real System	Simulator	Real System	Simulator
1.0	97.0%	96.8%	100.0%	100.0%
1.5	65.5%	65.1%	100.0%	100.0%
2.0	52.8%	51.0%	99.3%	99.3%
2.5	44.9%	46.1%	87.3%	88.3%
3.0	36.7%	38.3%	83.0%	84.1%
3.5	27.8%	28.0%	77.3%	77.0%
4.0	23.6%	24.1%	70.0%	68.9%

Table 2: Comparison of the SLO attainment reported by the simulator and the real system under different rates.

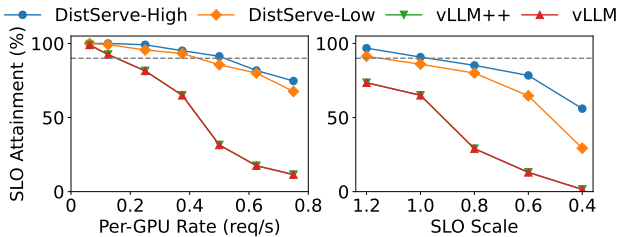


Figure 11: Ablation experiments.

6.4 Ablation Studies

We study the effectiveness of the two key innovations in DistServe: disaggregation and the placement searching algorithm. In §6.2, we choose the default parallelism setting for vLLM following its original paper [32]. So we implement "vLLM++" which enumerates different parallelism strategies and chooses the best. For DistServe, We also compare the placement found by Alg. 2 (DistServe-Low) with the one found by Alg. 1 (DistServe-High) which has fewer searching constraints and assumes high cross-node bandwidth. Since vLLM does not support inter-op parallelism and our physical testbed does not have high cross-node bandwidth, we use simulation for this experiment.

Simulator accuracy. Noticing that DNN model execution [24] has high predictability, even under parallel settings [33, 59]. We study the accuracy of the simulator in Tab. 2. For "vLLM" and "DistServe-Low", we compare the SLO attainment reported by the simulator and by real runs on our testbed under different rates. The error is less than 2% in all cases, verifying the accuracy of our simulator.

Results. Figure 11 shows the performance of the four systems when serving OPT-66B on the ShareGPT dataset. "vLLM++" has the same performance as "vLLM" because we find the default parallelism setting (intra-op=4) has the best per-GPU goodput. This further demonstrates the importance of disaggregation. The interference between the prefill and decoding phases significantly reduces the potential performance improvement through adjusting parallelism. In contrast, "DistLLM-High" can achieve further improvements over "DistLLM-Low" because it is not constrained by the deployment constraint that the prefill and decoding instance on one node should share the same model stage. Through disaggre-

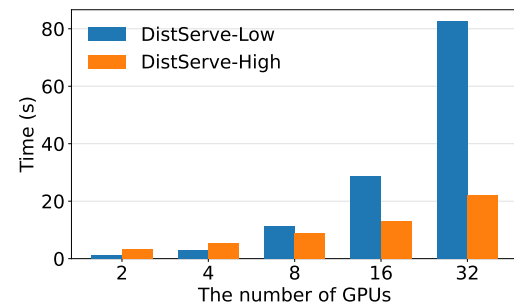


Figure 12: Algorithm Running Time

gation, we can use tailored parallelism strategies for prefill and decoding instances and optimize their targets without the coupling effects.

6.5 Algorithm Running Time

Figure 12 shows the running time for Alg. 1 (DistServe-Low) and Alg. 2 (DistServe-High) on an AWS m5d.metal instance with 96 cores as the number of GPUs ($N \times M$) provided to a single instance increases. According to the results, DistServe scales well with the number of GPUs and is independent of the model size. This is because the simulator only simulates discrete events and the running time is the same no matter how big the model is. On the other hand, both algorithms are highly parallelizable, as the searches for different parallelism strategies are independent of each other, allowing the execution time of the algorithms to accelerate almost linearly with more CPU cores.

As the number of GPUs increases, the execution time of "Dist-Low" becomes higher than that of "Dist-High". This is because the search for parallelism strategies for prefill and decoding instances in "Dist-High" is independent and can be parallelized. But for "Dist-Low", due to additional restrictions on deployment, we need to enumerate all the possible intra-node parallelism combinations for prefill and decoding instances. Even so, the execution time of the algorithm is in minutes, and since it only needs to be executed once before each redeployment, this overhead is acceptable.

7 Discussion

In this paper, we focus on the goodput-optimized setting and propose DistServe under the large-scale LLM serving scenario. As LLMs are widely used and deployed across various service scenarios with different optimization targets and resource limits, it becomes almost impossible to find a one-size-fits-all solution that effectively addresses all aspects of LLM serving. In this section, we discuss the pros and cons of DistServe and potentially better solutions in other scenarios.

Throughput-optimized scenarios. In offline applications that are not latency-sensitive, users typically have lower requirements for response time [45]. This allows serving systems to shift focus towards maximizing overall throughput instead of goodput and the effectiveness of DistServe may be

Rate (次/秒)	vLLM		DistServe-Low	
	真实系统	模拟器	真实系统	模拟器
1.0	97.0%	96.8%	100.0%	100.0%
1.5	65.5%	65.1%	100.0%	100.0%
2.0	52.8%	51.0%	99.3%	99.3%
2.5	44.9%	46.1%	87.3%	88.3%
3.0	36.7%	38.3%	83.0%	84.1%
3.5	27.8%	28.0%	77.3%	77.0%
4.0	23.6%	24.1%	70.0%	68.9%

表2: 在不同速率下, 模拟器与真实系统所报告的SLO达成率的对比。

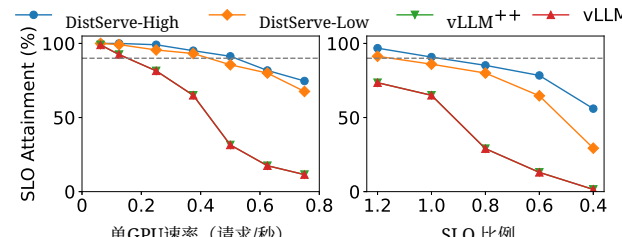


图 11: 消融实验。

6.4 消融研究

我们研究了 DistServe 中两项关键创新的有效性: 解耦以及放置方案搜索算法。在第 6.2 节中, 我们按照其原始论文 [32] 为 vLLM 选择默认的并行设置。因此, 我们实现了“vLLM++”, 该版本会枚举不同的并行策略并选择最佳策略。对于 DistServe, 我们将算法 2 (DistServe-Low) 找到的放置方案与算法 1 (DistServe-High) 找到的放置方案进行了比较, 后者具有更少的搜索约束, 并假设具有高的跨节点带宽。由于 vLLM 不支持算子间并行, 且我们的物理测试床不具备高的跨节点带宽, 因此我们使用仿真来进行此实验。

模拟器准确性。注意到DNN模型执行 [24] 具有很高的可预测性, 即使在并行 [33, 59]设置下也是如此。我们在表 2中研究了模拟器的准确性。对于“vLLM”和“DistServe-Low”, 我们比较了模拟器和真实系统在测试床上以不同速率运行时报告的SLO达成率。在所有情况下, 误差均小于2%, 验证了我们模拟器的准确性。

结果。图11展示了四个系统在ShareGPT数据集上服务 OPT-66B时的性能。“vLLM++”与“vLLM”的性能相同, 因为我们发现默认并行设置 (intra-op=4) 具有最佳的单GPU有效吞吐量。这进一步证明了解耦的重要性。预填充和解码阶段之间的干扰显著降低了通过调整并行性所带来的潜在性能提升。相比之下, "DistLLM-High"能够在"DistLLM-Low"的基础上实现进一步改进, 因为它不受部署约束的限制, 即同一节点上的预填充实例和解码实例必须共享相同的模型阶段。通过解耦,

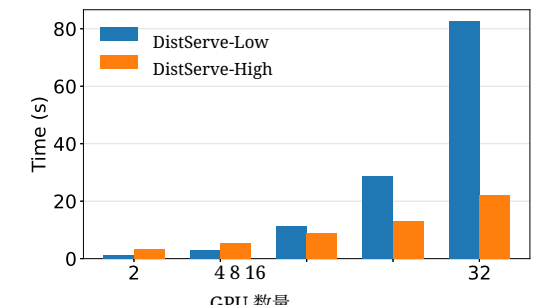


图 12: 算法运行时间

我们可以为预填充和解码实例采用定制化的并行策略, 并在没有耦合效应的情况下优化各自的目标。

6.5 算法运行时间

图 12 展示了当分配给单个实例的 GPU 数量 ($N \times M$) 增加时, 在具有 96 核的 AWS m5d.metal 实例上, 算法 1 (DistServe-Low) 和算法 2 (DistServe-High) 的运行时间。根据结果, DistServe 随着 GPU 数量的增加而良好扩展, 且与模型大小无关。这是因为模拟器仅模拟离散事件, 因此无论模型多大, 运行时间都相同。另一方面, 这两种算法都具有高度的并行性, 因为不同并行策略的搜索是相互独立的, 从而使得算法的执行时间能够随着 CPU 核心数量的增加而几乎线性加速。

随着GPU数量的增加, “Dist-Low”的执行时间超过了“Dist-High”。这是因为“Dist-High”中预填充和解码实例的并行策略搜索是相互独立的, 并且可以并行执行。然而, 对于“Dist-Low”, 由于部署上的额外限制, 我们需要枚举预填充和解码实例的所有可能节点内并行组合。即便如此, 算法的执行时间仅为几分钟, 且由于每次重新部署前只需执行一次, 因此这一开销是可以接受的。

7 讨论

本文聚焦于善吞吐量优化设置, 并在大规模大语言模型服务场景下提出了 DistServe。随着大语言模型在各种具有不同优化目标和资源限制的服务场景中被广泛使用和部署, 几乎不可能找到一个能够全面解决大语言模型服务所有方面的一刀切方案。在本节中, 我们讨论了 DistServe 的优缺点, 以及其他场景下可能更优的解决方案。

吞吐量优化场景。在那些非延迟敏感的离线应用中, 用户通常对响应时间的要求较低 [45]。这使得服务系统可以将重点从有效吞吐量和DistServe的有效性转移到最大化整体吞吐量上, 而DistServe的有效性可能会被

compromised. In this case, techniques such as chunked-prefill with piggyback [3, 9] may be preferred since it can fill each batch to the compute-bound threshold, thereby maintaining higher GPU utilization in every iteration.

Resource-constrained scenarios. Small-scale enterprises and individual researchers often lack the resources to deploy LLMs on large-scale clusters [45, 48]. In resource-constrained scenarios, such as environments with only a few or even a single GPU, the design space for DistServe is significantly limited. It struggles or even fails to adjust the parallel strategies and resource allocation to effectively enhance serving performance. In this case, simpler architectural choices like non-disaggregated systems [3, 32] may reduce deployment complexity and optimize operational efficiency.

Long-context scenarios. Nowadays, more and more GPT models support extremely long contexts, such as Claude-3 [11], Gemini-1.5 [22], and Large World Model (LWM) [34], which all have a 1M context window. In such scenarios, the transmission overhead will increase as the size of the KV cache grows linearly with the prompt length. However, the prefill computation grows quadratically, so the relative duration of transmission and prefill job decreases. Meanwhile, a longer context further exacerbates the disparity in computational demands between prefill and decoding jobs, leading to increased interference between them. Therefore, the disaggregation approach proposed in DistServe remains promising in long-context serving.

8 Related Work

Inference serving. There has been plenty of work on inference serving recently. They range from general-purpose production-grade systems like TorchServe [7] and NVIDIA Triton [19] to systems optimized specifically for Transformer-based LLMs [9, 18, 21, 33, 50, 53, 54, 60]. Among them, Orca [54] introduces continuous batching to increase throughput. vLLM [32] proposes paged-attention for fine-grained KV cache management. SARATHI [9] suggests a chunked-prefill approach, splitting a prefill request into chunks and piggybacking decoding requests to improve hardware utilization. FastServe [53] implements iteration-level preemptive scheduling to mitigate the queuing delay caused by long jobs. However, they all employ a colocation approach for prefill and decoding processing, thus leading to severe interference. There are also concurrent works such as Splitwise [38], Tetri-Infer [27] and DéjàVu [49] which adopt similar disaggregation idea to optimize LLM inference, further confirming the effectiveness of this method. Differently, DistServe emphasizes the goodput optimization scenario more and takes a closer look at the aspect of network bandwidth.

Goodput-optimized systems. Optimizing goodput is a hot topic in DL applications. Pollux [39] improves scheduling performance in DL clusters by dynamically adjusting resources for jobs to increase cluster-wide goodput. Sia [29]

introduces a heterogeneous-aware scheduling approach that can efficiently match cluster resources to elastic resource-adaptive jobs. Clockwork [23] and Shepherd [55] provide latency-aware scheduling and preemption to improve the serving goodput, but they only target traditional small models. AlpaServe [33] focuses on LLMs, employing model parallelism to statistically multiplex the GPU execution thus improving the resource utilization. However, it only targets the non-autoregressive generation. DistServe is the first work to optimize the goodput for autoregressive LLM inference.

Resource disaggregation. Resource disaggregated systems [17, 25, 43] decouple the hardware resources from the traditional monolithic server infrastructure and separate them into resource pools to manage independently. It allows for more flexible, efficient, and scalable deployment and increases resource utilization. Many applications benefit from a truly disaggregated data center with high-speed network bandwidth and heterogenous hardware support [12, 30, 57]. DistServe shares the concept by disaggregating its system components, allowing for independent resource scaling and management.

Model parallelism for training. DistServe is orthogonal to the large body of work on model parallelism in training [28, 36, 40, 46, 59]. As described in §3.3, inference-serving workloads have unique characteristics not found in training settings. Where these systems do intersect with DistServe, is in their methods for implementing model parallelism along various dimensions. DistServe can integrate new parallelism optimizations into its placement searching algorithm.

9 Conclusion

We present DistServe, a new LLM serving architecture that disaggregates the prefill and decoding computation. DistServe maximizes the per-gpu goodput – the maximum request rate that can be served adhering to the SLO attainment goal for each GPU provisioned, hence resulting in up to $7.4\times$ lower cost per LLM query with guaranteed satisfaction of SLOs. Our findings affirm that as latency becomes an increasingly important metric for LLM services, prefill and decoding disaggregation is a vital strategy in promising improved performance and service quality guarantees.

Acknowledgments. We sincerely thank our shepherd and the anonymous reviewers for their valuable feedback. This work was supported by the National Natural Science Foundation of China under the grant numbers 62172008, 62325201, and the National Natural Science Fund for the Excellent Young Scientists Fund Program (Overseas). Junda Chen is supported by UCSD fellowship and Hao Zhang is supported by UCSD faculty startup fund. Xin Jin is the corresponding author. Yinmin Zhong, Xuanzhe Liu, and Xin Jin are also with the Key Laboratory of High Confidence Software Technologies (Peking University), Ministry of Education.

损害。在这种情况下，可能更倾向于采用带捎带的分块预填充 [3, 9] 等技术，因为它可以将每个批处理填充至计算密集型阈值，从而在每次迭代中维持更高的GPU利用率。

资源受限场景。中小企业和个人研究人员通常缺乏在大规模集群上部署大语言模型的资源 [45, 48]。在资源受限的场景下，例如仅有少数甚至单 GPU 的环境，DistServe的设计空间受到极大限制。它难以甚至无法调整并行策略和资源分配来有效提升服务性能。在这种情况下，更简单的架构选择，如非分离式系统 [3, 32]，可以降低部署复杂性并优化运营效率。

长上下文场景。如今，越来越多的GPT模型支持极长上下文，例如Claude-3 [11], Gemini-1.5 [22], 和大世界模型（LWM）[34],，它们都具备100万词的上下文窗口。在此类场景中，随着提示长度增加，KV缓存的大小线性增长，传输开销也随之增加。然而，预填充计算呈二次方增长，因此传输和预填充任务的相对持续时间会减少。同时，更长的上下文进一步加剧了预填充和解码任务之间计算需求的差异，导致两者之间的干扰增加。因此，DistServe提出的解耦方法在长上下文服务中仍然具有广阔前景。

8 相关工作

推理服务。近年来，针对推理服务的研究工作层出不穷。这些工作涵盖了从TorchServe [7] 和NVIDIA Triton [19] 等通用生产级系统，到专为基于Transformer的大语言模型优化的系统 [9, 18, 21, 33, 50, 53, 54, 60]。其中，Orca [54] 引入连续批处理以提高吞吐量。vLLM [32] 提出了分页注意力机制以实现细粒度KV缓存管理。SARATHI [9] 提出了一种分块预填充方法，将预填充请求划分为分块，并搭载解码请求以提升硬件利用率。FastServe [53] 实现了迭代级抢占式调度，以缓解由长作业引起的排队延迟。然而，它们都采用共置方法进行预填充和解码处理，从而导致了严重干扰。此外，还有诸如Splitwise [38]、Tetris Infer [27] 和 DéjàVu [49] 等并行工作，它们采用类似的解耦理念来优化LLM推理，进一步证实了该方法的有效性。不同之处在于，DistServe更侧重于有效吞吐量优化场景，并更深入地考察了网络带宽这一方面。

有效吞吐量优化系统。优化有效吞吐量是深度学习应用中的一个热门话题。Pollux [39] 通过为作业动态调整资源来提高集群级有效吞吐量，从而提升了深度学习集群中的调度性能。Sia [29]

引入了一种异构感知调度方法，能够高效地将集群资源匹配到弹性的资源自适应作业。Clockwork [23] 和 Shepherd [55] 提供延迟感知调度和抢占功能，以提高服务有效吞吐量，但它们仅针对传统的小型模型。AlpaServe [33] 专注于大语言模型，采用模型并行对 GPU 执行进行统计复用，从而提高资源利用率。然而，它仅针对非自回归生成。DistServe是首个针对自回归大语言模型推理优化有效吞吐量的工作。

资源解耦。资源解耦系统 [17, 25, 43] 将硬件资源从传统的单体服务器基础设施中解耦出来，并将其分离为资源池以进行独立管理。这使得部署更加灵活、高效和可扩展，并提高了资源利用率。许多应用受益于具备高速网络带宽和异构硬件支持的真正解耦数据中心 [12, 30, 57]。DistServe 通过解耦其系统组件，共享这一概念，从而实现独立的资源扩展和管理。

训练中的模型并行。DistServe 与训练中的模型并行的大量现有工作相互正交 [28, 36, 40, 46, 59]。正如 §3.3 中所述，推理服务工作负载具有训练场景中未见过的独特特性。这些系统与 DistServe 的交叉点在于它们在沿各个维度实施模型并行性的方法。DistServe 可以将其放置搜索算法与新的并行性优化相结合。

9 结论

我们提出了 DistServe，这是一种新型的大语言模型服务架构，它将预填充和解码计算解耦。DistServe 最大化了每 GPU 有效吞吐量——即在为每个配置的 GPU 遵守 SLO 达成目标的前提下，可服务的最大请求速率，从而在保证满足服务等级目标的情况下，使每个大语言模型查询的成本降低最多 $7.4\times$ 。我们的研究结果证实，随着延迟成为大语言模型服务中日益重要的指标，预填充和解码解耦是一种关键策略，有望提升性能并保证服务质量。

致谢。我们衷心感谢我们的 Shepherd 以及匿名审稿人提供的宝贵反馈。本研究得到了中国国家自然科学基金的资助，资助编号为62172008和62325201，并得到了国家自然科学基金优秀青年科学家基金项目（海外）的资助。陈俊达由加州大学圣地亚哥分校奖学金资助，张浩由加州大学圣地亚哥分校教师启动资金资助。金鑫为通讯作者。钟寅民、刘炫德和金鑫还隶属于教育部高可信软件技术重点实验室（北京大学）。

References

[1] Introducing chatgpt. <https://openai.com/blog/chatgpt>, 2022.

[2] Bard, an experiment by google. <https://bard.google.com/>, 2023.

[3] Deepspeed model implementations for inference (mii), 2023.

[4] Inflection tech memo. <https://inflection.ai/assets/Inflection-1.pdf>, 2023.

[5] Lanchain usecase: Summarization, 2023.

[6] Nvidia collective communications library (nccl), 2023.

[7] Serve, optimize and scale pytorch models in production, 2023.

[8] Sharegpt teams. <https://sharegpt.com/>, 2023.

[9] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S Gulavani, and Ramachandran Ramjee. Sarathi: Efficient llm inference by piggy-backing decodes with chunked prefills. *arXiv preprint arXiv:2308.16369*, 2023.

[10] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints, 2023.

[11] Anthropic. Introducing the next generation of claude. <https://www.anthropic.com/news/claude-3-family>, 2024.

[12] Andrew Audibert, Yang Chen, Dan Graur, Ana Klimovic, Jiri Simsa, and Chandramohan A. Thekkath. A case for disaggregation of ml data processing, 2022.

[13] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. Longbench: A bilingual, multitask benchmark for long context understanding, 2023.

[14] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain,

William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. 2021.

[15] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[16] Wei-Lin Chiang, Zhuohan Li, Zi Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E. Gonzalez, Ion Stoica, and Eric P. Xing. Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality, 2023.

[17] Compute Express Link Consortium. Compute express link, 2023. Accessed: 2023-12-07.

[18] NVIDIA Corporation. Fastertransformer, 2019.

[19] NVIDIA Corporation. Triton inference server: An optimized cloud and edge inferencing solution., 2019.

[20] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness, 2022.

[21] Jiarui Fang, Yang Yu, Chengduo Zhao, and Jie Zhou. Turbotransformers: an efficient gpu serving system for transformer models. In *ACM PPOPP*, 2021.

[22] Google. Our next-generation model: Gemini 1.5. <https://deepmind.google/technologies/gemini/>, 2024.

[23] Arpan Gujarati, Reza Karimi, Safya Alzayat, Wei Hao, Antoine Kaufmann, Ymir Vigfusson, and Jonathan Mace. Serving DNNs like clockwork: Performance predictability from the bottom up. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 443–462. USENIX Association, November 2020.

[24] Arpan Gujarati, Reza Karimi, Safya Alzayat, Wei Hao, Antoine Kaufmann, Ymir Vigfusson, and Jonathan Mace. Serving DNNs like clockwork: Performance predictability from the bottom up. In *USENIX OSDI*, 2020.

[25] Zhiyuan Guo, Zijian He, and Yiyang Zhang. Mira: A program-behavior-guided far memory system. In *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP '23*, page 692–708, New York, NY, USA, 2023. Association for Computing Machinery.

参考文献

[1]Introducing ChatGPT. <https://openai.com/blog/chatgpt>, 2022.

[2] Bard, 谷歌的一项实验。 <https://bard.google.com/>, 2023年。

[3] 用于推理的DeepSpeed模型实现（MII）, 2023年。

[4] Inflection技术备忘录。 <https://inflection.ai/assets/Inflection-1.pdf>, 2023年。

[5] LangChain用例：摘要生成, 2023年。

[6] 英伟达集合通信库（NCCL）, 2023年。

[7] Serve, optimize and scale PyTorch models in production, 2023.

[8] ShareGPT 团队。 <https://sharegpt.com/>, 2023。

[9] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S Gulavani, and Ramachandran Ramjee. Sarathi: 通过分块预填充搭载解码以实现高效 LLM推理。arXiv 预印本 arXiv:2308.16369, 2023。

[10] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy,Federico Lebrón, and Sumit Sanghai. GQA: 从多头检查点训练广义多查询变换器模型, 2023。

[11]Anthropic. 介绍下一代 Claude。 <https://www.anthropic.com/news/claude-3-family>, 2024。

[12]Andrew Audibert, Yang Chen, Dan Graur, Ana Klimovic, Jiri Simsa,and Chandramohan A. Thekkath. 机器学习数据处理解耦的必要性, 2022。

[13] 白宇诗, 吕欣, 张家杰, 吕洪昌, 唐建凯, 黄指点, 杜正晓, 刘泉, 曾奥涵, 侯雷, 董宇笑, 唐杰, 和李姣子。LongBench：面向长上下文理解的双语多任务基准, 2023。

[14] 马克·陈, 杰里·托雷克, 崔惠雨, 袁启明, 恩里克·庞德·德·奥利维拉·平托, 贾里德·卡普兰, 哈里·爱德华兹, 尤里·布达, 尼古拉斯·约瑟夫, Greg Brock-man, 亚历克斯·雷, 劳尔·普里, 格蕾琴·克鲁格, 迈克尔·彼得罗夫, 海迪·赫拉夫, 吉里什·萨斯特里, 帕梅拉·米什金, 布鲁克·陈, 斯科特·格雷, 尼克·赖德, 米哈伊尔·帕夫洛夫, 阿莱西亚·鲍尔, 卢卡斯·凯泽, 穆罕默德·巴瓦里安, 克莱门斯·温特, 菲利普·蒂莱, 费利佩·佩特罗斯基·苏赫, 戴夫·卡明斯, 马蒂亚斯·普拉珀特, 弗提奥斯·昌齐斯, 伊丽莎白·巴恩斯, 亚里尔·赫伯特·沃斯, William Heb-gen Guss, 亚历克斯·尼科尔, 亚历克斯·帕伊诺, 尼古拉斯·特拉克, 唐杰, 伊戈尔·巴布什金, 苏希尔·巴拉吉, 尚塔努·贾因,

威利 am Saunders, Christopher Hese, Andrew N. C ese,, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 评估在代码上训练的大型语言模型。2021。[15] 马克·陈, 杰里·托雷克, 崔惠雨, 袁启明, 恩里克·庞德·德·奥利维拉·平托, 贾里德·卡普兰, 哈里·爱德华兹, 尤里·布达, 尼古拉斯·约瑟夫, 格雷格·布罗克曼等。评估在代码上训练的大型语言模型。arXiv 预印本 arXiv:2107.03374, 2021。

[16] Wei-Lin Chiang, 李卓涵, Zi Lin, 盛盈, Zhanghao Wu, 张浩, 郑连民, 庄思远, Yonghao Zhuang, 约瑟夫·E·冈萨雷斯, Ion Sto- ica, 星忆捷. Vicuna: 一个以90%* ChatGPT 质量令人惊艳 GPT-4 的开源聊天机器人, 2023.

[17] Compute Express Link 联盟. Compute Express Link, 2023. 访问时间: 2023-12-07.

[18] 英伟达公司. Fastertransformer, 2019.

[19] 英伟达公司. Triton 推理服务器: 一种优化的云和边缘推理解决方案., 2019.

[20] Tri Dao, 傅丹尼尔, Stefano Ermon, Atri Rudra, 克里斯托弗·雷. FlashAttention: 一种具有 IO 感知的高效且内存高效的精确注意力, 2022.

[21] Jiarui Fang, Yang Yu, Chengduo Zhao, Jie Zhou. Turbotransformers: 一种用于 Transformer 模型的高效 GPU 服务系统. In ACM PPOPP, 2021.

[22]谷歌。我们的下一代模型： Gemini 1.5。 <https://deepmind.google/technologies/gemini/>, 2024.

[23] Arpan Gujarati, Reza Karimi, Safya Alzayat, Wei Hao, Antoine Kaufmann, Ymir Vigfusson, and Jonathan Mace. 像时钟一样精确地服务 DNN：自下而上的性能可预测性。In 14th USENIX Symposium on Operating Systems Design and Implementa- tion (OSDI 20), pages 443–462. USENIX 协会, November 2020.

[24] Arpan Gujarati, Reza Karimi, Safya Alzayat, Wei Hao, Antoine Kaufmann, Ymir Vigfusson, and Jonathan Mace. 像时钟一样精确地服务 DNN：自下而上的性能可预测性。In USENIX OSDI, 2020.

[25] Zhiyuan Guo, Zijian He, 和 张一颖. Mira： 一个程序行为引导的远内存系统。收录于第29届操作系统原理研讨会（SOSP '23）论文集, 第692–708页, 美国纽约, 2023年。计算机协会。

[26] Mingcong Han, Hanze Zhang, Rong Chen, and Haibo Chen. Microsecond-scale preemption for concurrent GPU-accelerated DNN inferences. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 539–558, Carlsbad, CA, July 2022. USENIX Association.

[27] Cunchen Hu, Heyang Huang, Liangliang Xu, Xusheng Chen, Jiang Xu, Shuang Chen, Hao Feng, Chenxi Wang, Sa Wang, Yungang Bao, Ninghui Sun, and Yizhou Shan. Inference without interference: Disaggregate llm inference for mixed downstream workloads, 2024.

[28] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Mia Xu Chen, Dehao Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V. Le, Yonghui Wu, and Zhifeng Chen. Gpipe: Efficient training of giant neural networks using pipeline parallelism, 2019.

[29] Suhas Jayaram Subramanya, Daiyaan Arfeen, Shouxu Lin, Aurick Qiao, Zhihao Jia, and Gregory R Ganger. Sia: Heterogeneity-aware, goodput-optimized ml-cluster scheduling. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 642–657, 2023.

[30] Xin Jin, Zhihao Bai, Zhen Zhang, Yibo Zhu, Yinmin Zhong, and Xuanzhe Liu. Distmind: Efficient resource disaggregation for deep learning workloads. *IEEE/ACM Transactions on Networking*, pages 1–16, 2024.

[31] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.

[32] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention, 2023.

[33] Zhuohan Li, Lianmin Zheng, Yinmin Zhong, Vincent Liu, Ying Sheng, Xin Jin, Yanping Huang, Zhifeng Chen, Hao Zhang, Joseph E Gonzalez, et al. Alpaserve: Statistical multiplexing with model parallelism for deep learning serving. *arXiv*, 2023.

[34] Hao Liu, Wilson Yan, Matei Zaharia, and Pieter Abbeel. World model on million-length video and language with ringattention. *arXiv*, 2024.

[35] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. Ray: A distributed framework for emerging AI applications. In *USENIX OSDI*, 2018.

[36] Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil R. Devanur, Gregory R. Ganger, Phillip B. Gibbons, and Matei Zaharia. Pipedream: Generalized pipeline parallelism for dnn training. In *ACM SOSP*, 2019.

[37] OpenAI. Gpt-4 technical report, 2023.

[38] Pratyush Patel, Esha Choukse, Chaojie Zhang, Íñigo Goiri, Aashaka Shah, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative llm inference using phase splitting, 2023.

[39] Aurick Qiao, Sang Keun Choe, Suhas Jayaram Subramanya, Willie Neiswanger, Qirong Ho, Hao Zhang, Gregory R. Ganger, and Eric P. Xing. Pollux: Co-adaptive cluster scheduling for goodput-optimized deep learning. In *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*, pages 1–18. USENIX Association, July 2021.

[40] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. Zero: Memory optimizations toward training trillion parameter models, 2020.

[41] Reuters, 2023.

[42] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, et al. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.

[43] Yizhou Shan, Yutong Huang, Yilun Chen, and Yiying Zhang. {LegoOS}: A disseminated, distributed {OS} for hardware resource disaggregation. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pages 69–87, 2018.

[44] Noam Shazeer. Fast transformer decoding: One write-head is all you need, 2019.

[45] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Daniel Y. Fu, Zhiqiang Xie, Beidi Chen, Clark Barrett, Joseph E. Gonzalez, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. Flexgen: High-throughput generative inference of large language models with a single gpu, 2023.

[46] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism, 2020.

[47] John F Shortle, James M Thompson, Donald Gross, and Carl M Harris. *Fundamentals of queueing theory*, volume 399. John Wiley & Sons, 2018.

[26] 韩铭聪, 张涵泽, 陈榕, 和 陈海波。面向并发GPU加速的深度学习推理的微妙级抢占。收录于第16届USENIX操作系统设计与实现研讨会 (OSDI 22) , 第539–558页, 加利福尼亚州卡尔斯巴德, 2022年7月。USENIX协会。

[27] 胡存辰, 黄和阳, 徐亮亮, 陈旭升, 徐江, 陈爽, 冯浩, 王晨曦, 王飒, 包云岗, 孙凝晖, 单奕舟。无干扰推理: 解耦大语言模型推理以处理混合下游工作负载, 2024。

[28] 黄艳萍, 程宇龙, 安库尔·巴帕纳, 奥尔汉·菲拉特, 陈美夏, 陈德浩, 李炫钟, 杨志全, 李飞飞, 吴勇辉, 陈智锋。Gpipe: 利用流水线并行高效训练巨型神经网络, 2019。

[29] 苏哈斯·贾亚拉姆·苏布拉马尼亚, 代扬·阿尔芬, 林书旭, 乔奥里克, 贾志豪, 格雷戈里·R·甘格。Sia: 异构感知、有效吞吐量优化的机器学习集群调度。载于第29届操作系统原理研讨会论文集, 第642–657页, 2023。

[30] 金鑫, 白志豪, 张震, 朱亦博, 钟寅民, 刘炫德。DistMind: 面向深度学习工作负载的高效资源解耦。IEEE/ACM网络汇刊, 页1–16, 2024。

[31] 权佑淑, 李卓涵, 庄思远, 盛盈, 郑连民, 余昊宇, Joseph E. Gonzalez, 张浩, 伊恩·斯托伊卡。使用PagedAttention进行大语言模型服务的高效内存管理。In Proceedings of the ACM SIGOPS第29届操作系统原理研讨会, 2023。

[32] 权佑淑, 李卓涵, 庄思远, 盛盈, 郑连民, 余昊宇, Joseph E. Gonzalez, 张浩, 伊恩·斯托伊卡。使用PagedAttention进行大语言模型服务的高效内存管理, 2023。

[33] 李卓涵, 郑连民, 钟寅民, 刘文森, 盛盈, 金鑫, 黄艳萍, 陈智锋, 张浩 , Joseph E Gonzalez, 等 。Alpaserve: 基于模型并行性的深度学习服务统计复用。arXiv, 2023。

[34] 刘浩, 颜威尔逊, 马泰·扎哈里亚, 和 皮特·阿比尔。基于环形注意力的百万长度视频与语言世界模型。arXiv, 2024。

[35] 菲利普·莫里茨, 罗伯特·西原, 斯蒂芬妮·王, 阿列克谢·图马诺夫, 理查德·刘, 埃里克·梁, Melih Elibol, 杨宗恒, 威廉·保罗, 迈克尔·I·乔丹, 以及伊恩·斯托伊卡。Ray: 面向新兴人工智能应用的分布式框架。载于 USENIX OSDI, 2018 年。

[36] 迪帕克·纳拉扬, 亚伦·哈拉普, 阿马尔·法尼沙伊, 维韦克·塞沙德里, 尼基尔·R·德瓦努尔, 格雷戈里·R·甘格, 菲利普·B·吉本斯, 以及马泰·扎哈里亚。Pipedream: 面向 DNN 训练的通用流水线并行。载于 ACM SOSP, 2019 年。

[37] OpenAI. GPT-4 技术报告, 2023.

[38] 普拉图什·帕特尔, 艾莎·乔克瑟, 章超杰, Íñigo Goiri, 阿莎卡·沙阿, 萨德·马莱基, 和 Ricardo Bianchini. Splitwise: 使用阶段拆分的生成式 LLM 推理, 2023.

[39] 乔奥里克, 崔相根, Suhas Jayaram Subramanya, 威利·奈斯万格, 何启荣, 张浩, Gregory R. Ganger, 和 星忆捷。Pollux: 用于有效吞吐量优化的深度学习协同自适应集群调度。In 第 15 届 USENIX 操作系统设计与实现研讨会 (OSDI 21), 第 1–18 页。USENIX 协会, 2021 年 7 月。

[40] 萨迈亚姆·拉杰班达里, 杰夫·拉瑟利, 奥拉通吉·鲁瓦塞, 和何旭雄。Zero: 面向训练万亿参数模型的内存优化, 2020。

[41] 路透社, 2023。

[42] 巴普蒂斯特·罗齐尔, 约纳斯·盖林, 法比安·格洛克勒, 斯滕·苏特拉, 伊泰·加特, 谭晓青, 约西·阿迪, 刘靖宇, 塔尔·雷梅斯, 杰雷米·拉潘, 等人。Code Llama: 代码的开放基础模型。arXiv 预印本 arXiv:2308.12950, 2023。

[43] 单奕舟, 黄雨桐, 陈依伦, 和张一颖。{LegoOS}: 用于硬件资源解聚的分布式、分散式 {OS}。载于第十三届 USENIX 操作系统设计与实现研讨会 (OSDI 18), 第 69–87 页, 2018。

[44] 诺姆·沙泽尔。快速 Transformer 解码: 单写头足矣, 2019。

[45] 盛盈,郑连民,袁兵航, 李卓涵, 马克斯·里亚宾宁, 傅丹尼尔, 谢志强, 陈贝迪, 克拉克·巴雷特, 约瑟夫·E·冈萨雷斯, 梁佩齐, 克里斯托弗·雷, 伊恩·斯托伊卡, 张策。FlexGen: 使用单 GPU 对大型语言模型进行高吞吐量生成式推理, 2023。

[46] 穆罕默德·肖伊比, 莫斯托法·帕特瓦里, 劳尔·普里, 帕特里克·勒格雷斯特利, 贾里德·卡斯珀, 布赖恩·卡坦扎罗。Megatron-lm: 使用模型并行训练数十亿参数语言模型, 2020。

[47] 约翰·F·肖特尔, 詹姆斯·M·汤普森, 唐纳德·格罗斯, 卡尔·M·哈里斯。《排队论基础》, 第399卷。约翰·威利儿子出版社, 2018。

15

15

翻译服务由 ChatECNU 大语言模型平台提供 <https://chat.ecnu.edu.cn/html/>
技术方案基于 PDFMathTranslate-next 项目 <https://github.com/PDFMathTranslate/PDFMathTranslate-next>

[48] Yixin Song, Zeyu Mi, Haotong Xie, and Haibo Chen. Powerinfer: Fast large language model serving with a consumer-grade gpu, 2023.

[49] Foteini Strati, Sara Mcallister, Amar Phanishayee, Jakub Tarnawski, and Ana Klimovic. DéjàVu: Kv-cache streaming for fast, fault-tolerant generative llm serving, 2024.

[50] Yiming Su, Chengcheng Wan, Utsav Sethi, Shan Lu, Madan Musuvathi, and Suman Nath. Hotgpt: How to make software documentation more useful with a large language model? In *Proceedings of the 19th Workshop on Hot Topics in Operating Systems*, pages 87–93, 2023.

[51] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. Llama: Open and efficient foundation language models, 2023.

[52] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Neural Information Processing Systems*, 2017.

[53] Bingyang Wu, Yinmin Zhong, Zili Zhang, Gang Huang, Xuanzhe Liu, and Xin Jin. Fast distributed inference serving for large language models. *arXiv preprint arXiv:2305.05920*, 2023.

[54] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soo-jeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for {Transformer-Based} generative models. In *USENIX OSDI*, 2022.

[55] Hong Zhang, Yupeng Tang, Anurag Khandelwal, and Ion Stoica. Shepherd: Serving dnns in the wild. 2023.

[56] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, Todor Mihaylov, Myle Ott, Sam Shleifer, Kurt Shuster, Daniel Simig, Punit Singh Koura, Anjali Sridhar, Tianlu Wang, and Luke Zettlemoyer. Opt: Open pre-trained transformer language models, 2022.

[57] Yiyi Zhang. Make it real: An end-to-end implementation of a physically disaggregated data center. *SIGOPS Oper. Syst. Rev.*, 57(1):1–9, jun 2023.

[58] Kai Zhao, Sheng Di, Sihuan Li, Xin Liang, Yujia Zhai, Jieyang Chen, Kaiming Ouyang, Franck Cappello, and Zizhong Chen. Ft-cnn: Algorithm-based fault tolerance for convolutional neural networks. *IEEE Transactions on Parallel and Distributed Systems*, 32(7):1677–1689, 2021.

[59] Lianmin Zheng, Zhuohan Li, Hao Zhang, Yonghao Zhuang, Zhifeng Chen, Yanping Huang, Yida Wang, Yuanzhong Xu, Danyang Zhuo, Eric P. Xing, Joseph E. Gonzalez, and Ion Stoica. Alpa: Automating inter- and Intra-Operator parallelism for distributed deep learning. In *USENIX OSDI*, 2022.

[60] Zhe Zhou, Xuechao Wei, Jiejing Zhang, and Guangyu Sun. {PetS}: A unified framework for {Parameter-Efficient} transformers serving. In *USENIX ATC*, 2022.

[48] Yixin Song, Zeyu Mi, Haotong Xie, 陈海波。PowerInfer：使用消费级GPU实现快速大语言模型服务，2023。[49] Foteini Strati, Sara Mcallister, 阿马尔·法尼沙伊, Jakub Tarnawski, Ana Klimovic. DéjàVu：通过KV缓存流实现快速、容错的生成式大语言模型服务，2024。[50] Yiming Su, Chengcheng Wan, Utsav Sethi, Shan Lu, Madan Musuvathi, Suman Nath。HotGPT：如何利用大语言模型让软件文档更有用？收录于第19届操作系统热点话题研讨会论文集，第87–93页，2023。[51] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, Guillaume Lample。LLaMA：开放且高效的基础语言模型，2023。[52] Ashish Vaswani, 诺姆·沙泽尔, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, Illia Polosukhin。你只需关注。神经信息处理系统，2017。[53] Bingyang Wu, 钟寅民, Zili Zhang, Gang Huang, 刘炫德, 金鑫。面向大型语言模型的快速分布式推理服务。arXiv 预印本 arXiv:2305.05920，2023。[54] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soo-jeong Kim, Byung-Gon Chun。Orca：一个用于 {基于Transformer的} 生成式模型的分布式服务系统。USENIX OSDI，2022。

[55] Hong Zhang, Yupeng Tang, Anurag Khandelwal, and 伊恩·斯托伊卡. Shepherd: Serving dnns in the wild. 2023.[56] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher De- wan, Mona Diab, Xian Li, Xi Victoria Lin, Todor Mi- haylov, Myle Ott, Sam Shleifer, Kurt Shuster, Daniel Simig, Punit Singh Koura, Anjali Sridhar, Tianlu Wang, and Luke Zettlemoyer. Opt: Open pre-trained trans- former language models, 2022.[57] 张一颖. 使其成真: 一个物理解耦数据中心的端到端实现. SIGOPS Oper. Syst. Rev., 57(1):1–9, jun 2023.[58] Kai Zhao, Sheng Di, Sihuan Li, Xin Liang, Yujia Zhai, Jieyang Chen, Kaiming Ouyang, Franck Cappello, and Zizhong Chen. ft-cnn: 基于算法的容错卷积神经网络. IEEE Transactions on Parallel and Distributed Systems, 32(7):1677–1689, 2021.[59] 郑连民, 李卓涵, 张浩, Yonghao Zhuang, 陈智锋, 黄艳萍, Yida Wang, Yuanzhong Xu, Danyang Zhuo, 星忆捷, 约瑟夫·E·冈萨雷斯, and 伊恩·斯托伊卡. Alpa: 自动实现分布式深度学习的算子间和算子内并行. In USENIX OSDI, 2022.[60] Zhe Zhou, Xuechao Wei, Jiejing Zhang, and Guangyu Sun. {PetS}: 用于 {参数高效} Transformer服务的统一框架. In USENIX ATC, 2022.

A Latency Model for LLM Inference

To accurately simulate the goodput of different placement strategies, we use an analytical model to predict the execution time of the prefill and decoding phases in LLM inference.

In modern LLM serving systems [18, 32, 53], memory-bound operations like Softmax and LayerNorm are usually fused with matrix multiplication kernels for efficiency. Thus the GEMMs dominate the overall latency and our analysis primarily focuses on them.

A.1 Symbol Definition

Here are symbols related to the architecture of the model:

- h : hidden size
- n : number of heads
- s : head size ($h = n \cdot s$)
- m : FFN intermediate size

Note: If tensor parallelism is used, h , n , and m should be divided by the tensor parallelism size.

Below are symbols that characterize the batch to be executed:

- B : batch size
- $l_0, l_1, \dots, l_{B-1}$: input length of each request within the batch
- t : number of tokens in the batch, ($t = \sum_{i=0}^{B-1} l_i$)
- t_2 : squared sum of the input lengths ($t_2 = \sum_{i=0}^{B-1} l_i^2$)
- b : block size in the attention kernel. This parameter is used in FlashAttention [20], a common kernel optimization technique adopted by current LLM serving systems.

A.2 Prefill Phase Latency Modeling

Since the attention operation uses specially optimized kernels, we first discuss the other four matrix multiplications in the prefill phase:

GEMM Name	Shape of M	Shape of N
QKV Linear	(t, h)	$(h, 3h)$
Attn Output	(t, h)	(h, h)
FFN Input	(t, h)	(h, m)
FFN Output	(t, m)	(m, h)

The arithmetic intensity (AI) of these operations is $O(t)$. On NVIDIA A100-80GB GPU, it is compute-bound when AI is over 156. Since t usually can reach several hundred in real cases, all of these operations are compute-bound. Therefore, we can model the latency of these operations according to the total FLOPs:

$$T_1 = C_1 \cdot (4th^2 + 2thm)$$

Next, we discuss the prefill attention operation with FlashAttention [20] optimization. Since the attention only operates among the tokens in the same request, current implementations launch attention kernels for each request in the same batch. For one attention head and a request with

l tokens, the attention kernel needs to perform a total of $2sl + 3sl \cdot (l/b) \approx 3sl \cdot (l/b)$ memory reads and writes, alongside $2sl^2 + sl(l/b) \approx 2sl^2$ FLOPs. So the AI is $2b/3 = 10.677$ (when $b = 16$) or 21.333 (when $b = 32$), indicating that it is a memory-bound operation on A100 GPU. Therefore, the whole attention layer latency (including all requests and all heads) can be modeled as:

$$T_2 = C_2 \cdot n \cdot \sum_{i=0}^{B-1} \frac{3sl_i^2}{b} = C_2 \cdot \frac{3nst_2}{b} = C_2 \cdot \frac{3ht_2}{b}$$

Overall, the latency of the prefill phase can be modeled as:

$$T_{Prefill} = C_1 \cdot (4th^2 + 2thm) + C_2 \cdot \frac{3ht_2}{b} + C_3$$

We use C_3 to quantify other overheads like Python Runtime, system noise, and so on. Then we use profiling and interpolation to figure out the values of C_1 , C_2 , and C_3 .

A.3 Decoding Phase Latency Modeling

Similarly, we first focus on the following GEMMs in the decoding phase:

GEMM Name	Shape of M	Shape of N
QKV Linear	(B, h)	$(h, 3h)$
Attn Output	(B, h)	(h, h)
FFN Input	(B, h)	(h, m)
FFN Output	(B, m)	(m, h)

The AI of these operations is $O(B)$. B is limited by the GPU memory size and stringent latency requirements, so in existing serving scenarios, these operations are memory-bound. The total memory reads and writes is $8Bh + 4h^2 + 2hm + 2Bm$, and since h and m are usually significantly larger than B , we can model the latency as:

$$T_3 = C_4 \cdot (4h^2 + 2hm)$$

As for the decoding attention operation, for one attention head and a request with l generated tokens, it needs to perform $3sl$ memory reads and writes, alongside $2sl$ FLOPs. It is memory-bound, so we can model the latency of decoding attention as:

$$T_4 = C_5 \cdot n \cdot 3s \sum_{i=0}^{B-1} l_i = C_5 \cdot 3ht$$

Summing up, the latency of the decoding phase is:

$$T_{Decoding} = C_4 \cdot (4h^2 + 2hm) + C_5 \cdot 3ht$$

Here we do not introduce the overhead term (like C_3 in the profiling stage) because $4h^2 + 2hm$ is already a constant, and the overhead can be put into C_4 . Similarly, we use profiling and interpolation to figure out the values of C_4 and C_5 .

LLM推理的延迟模型

为了准确模拟不同放置策略的有效吞吐量，我们使用分析模型来预测LLM推理中预填充阶段和解码阶段的执行时间。

在现代大语言模型服务系统中 [18, 32, 53], , 像 Softmax和层归一化这样的内存受限操作通常与矩阵乘法内核融合以提升效率。因此，通用矩阵乘法主导了整体延迟，我们的分析主要关注它们。

A.1 符号定义

以下是与模型架构相关的符号：

- h**: 隐藏层大小
 - n**: 注意力头数
 - s**: 头大小 (**h** = **n** · **s**)
 - m**: 前馈网络中间层大小
- 注意：如果使用张量并行，**h**、**n** 和 **m** 应除以张量并行度。

以下是用于表征待执行批处理的符号：

- B**: 批大小
- l0, l1, ..., lB-1**: 批处理内每个请求的输入长度

- t**: 批处理中的令牌数量, (**t** = $\sum_{i=0}^{B-1} l_i$)
- t2**: 输入长度的平方和 (**t2** = $\sum_{i=0}^{B-1} l_i^2$)
- b**: 注意力内核中的块大小。该参数用于 FlashAttention [20], 一种当前大语言模型服务系统所采用的常见内核优化技术。

A.2 预填充阶段延迟建模

由于注意力操作使用了特别优化的内核，我们首先讨论预填充阶段中的其他四个矩阵乘法：

GEMM 名称	M的形状	N 的形状
QKV线性层	(t, h)	(h, 3h)
注意力输出	(t, h)	(h, h)
FFN 输入	(t, h)	(h, m)
FFN 输出	(t, m)	(m, h)

这些操作的算术强度（AI）为 $O(t)$ 。在 NVIDIA A100-80GB GPU 上，当 AI 超过 156 时即为计算受限。由于 **t** 在实际场景中通常可达数百，因此这些操作均为计算受限。因此，我们可以根据总 FLOPs 来对这些操作的延迟进行建模：

$$T_1 = C_1 \cdot (4th^2 + 2thm)$$

接下来，我们讨论带有 FlashAttention [20] 优化的预填充注意力操作。由于注意力仅在相同请求的令牌之间操作，当前的实现在同一批处理中为每个请求启动注意力核。对于某个注意力头和具有

1 个令牌请求，注意力核需要执行总共 $2sl + 3sl \cdot (l/b) \approx 3sl \cdot (l/b)$ 次内存读写，以及 $2sl^2 + sl(l/b) \approx 2sl^2$ FLOPs。因此，人工智能（AI）值为 $2b/3 = 10.677$ （当 **b** = 16 时）或 21.333（当 **b** = 32 时），这表明它在 A100 GPU 上是一个内存受限的操作。因此，整个注意力层延迟（包括所有请求和所有注意力头）可以建模为：

$$T_2 = C_2 \cdot n \cdot \sum_{i=0}^{B-1} \frac{3sl_i^2}{b} = C_2 \cdot \frac{3nst_2}{b} = C_2 \cdot \frac{3ht_2}{b}$$

总体而言，预填充阶段的延迟可以建模为：

$$T_{Prefill} = C_1 \cdot (4th^2 + 2thm) + C_2 \cdot \frac{3ht_2}{b} + C_3$$

我们使用 C_3 来量化其他开销，如 Python 运行时、系统噪声等。然后我们使用性能分析和插值来确定 C_1 , C_2 , 和 C_3 的值。

A.3 解码阶段延迟建模

类似地，我们首先关注解码阶段中的以下通用矩阵乘法：

GEMM 名称	M 的形状	N 的形状
QKV线性层	(B, h)	(h, 3h)
注意力输出	(B, h)	(h, h)
FFN 输入	(B, h)	(h, m)
FFN 输出	(B, m)	(m, h)

这些操作的人工智能为 $O(B)$ 。**B** 受限于 GPU 内存大小和严格的延迟要求，因此在现有的服务场景中，这些操作是内存受限的。总的内存读写为 $8Bh + 4h^2 + 2hm + 2Bm$ ，并且由于 **h** 和 **m** 通常显著大于 **B**，我们可以将延迟建模为：

$$T_3 = C_4 \cdot (4h^2 + 2hm)$$

至于解码注意力操作，对于一个注意力头和一个具有 1 个生成令牌请求，它需要执行 $3sl$ 次内存读写，以及 $2sl$ FLOPs。由于它是内存受限的，我们可以将解码注意力的延迟建模为：

$$T_4 = C_5 \cdot n \cdot 3s \sum_{i=0}^{B-1} l_i = C_5 \cdot 3ht$$

综上所述，解码阶段的延迟为：

$$T_{Decoding} = C_4 \cdot (4h^2 + 2hm) + C_5 \cdot 3ht$$

这里我们没有引入开销项（如₃ 在分析阶段中的那样），因为 $4h^2 + 2hm$ 已经是一个常数，开销可以并入 C_4 。同样，我们使用性能分析和插值来确定 C_4 和 C_5 的值。

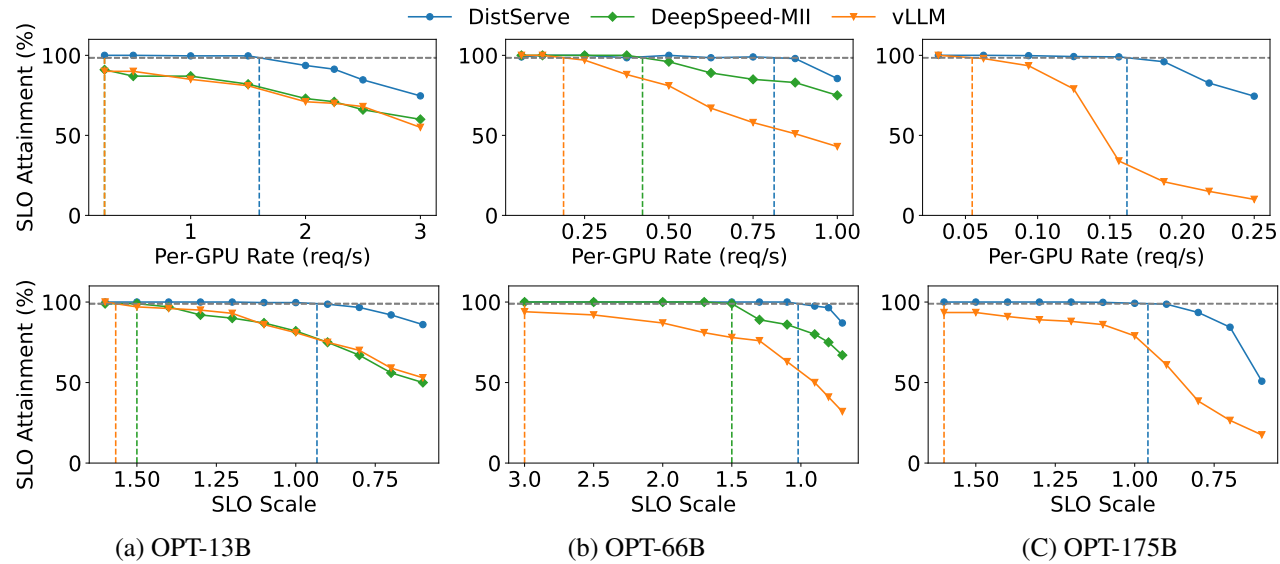


Figure 13: Chatbot application with OPT models on the ShareGPT dataset.

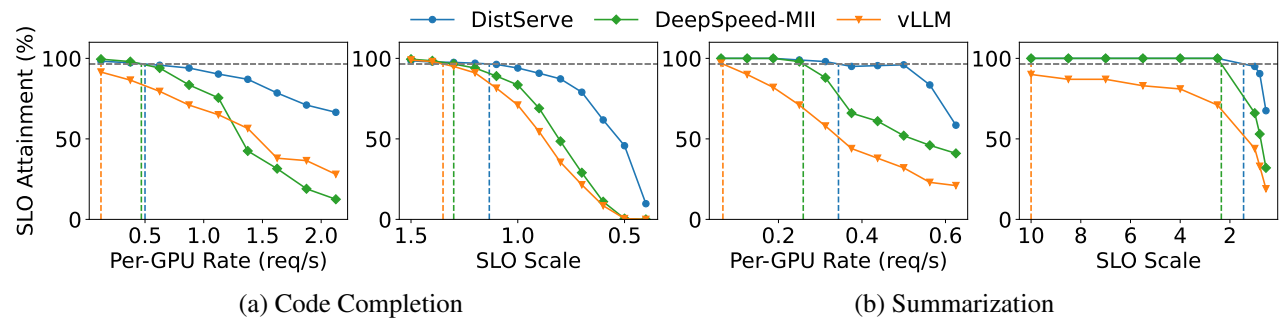


Figure 14: Code completion and summarization tasks with OPT-66B on HumanEval and LongBench datasets, respectively.

B DistServe Placements in End-to-end Experiments

Table 3 shows the tensor parallelism (TP) and pipeline parallelism (PP) configurations for prefill and decoding instances chosen by DistServe in the end-to-end experiments §6.2.

Model	Dataset	Prefill		Decoding	
		TP	PP	TP	PP
OPT-13B	ShareGPT	2	1	1	1
OPT-66B	ShareGPT	4	1	2	2
OPT-66B	LongBench	4	1	2	2
OPT-66B	HumanEval	4	1	2	2
OPT-175B	ShareGPT	3	3	4	3

Table 3: The parallelism strategies chosen by DistServe in the end-to-end experiments.

C End-to-end Results under 99% SLO attainment

Figure 13 and Figure 14 show the end-to-end performance between DistServe and baselines with the same setup in §6.2 except that the SLO attainment goal is changed to 99%. We can see that under a more stringent SLO attainment goal, compared to vLLM, DistServe can still sustain $3\times-8\times$ higher rate and $1.24\times-6.67\times$ more stringent SLO. When compared to DeepSpeed-MII, DistServe can achieve $1.32\times-8\times$ higher rate and $1.20\times-1.58\times$ more stringent SLO.

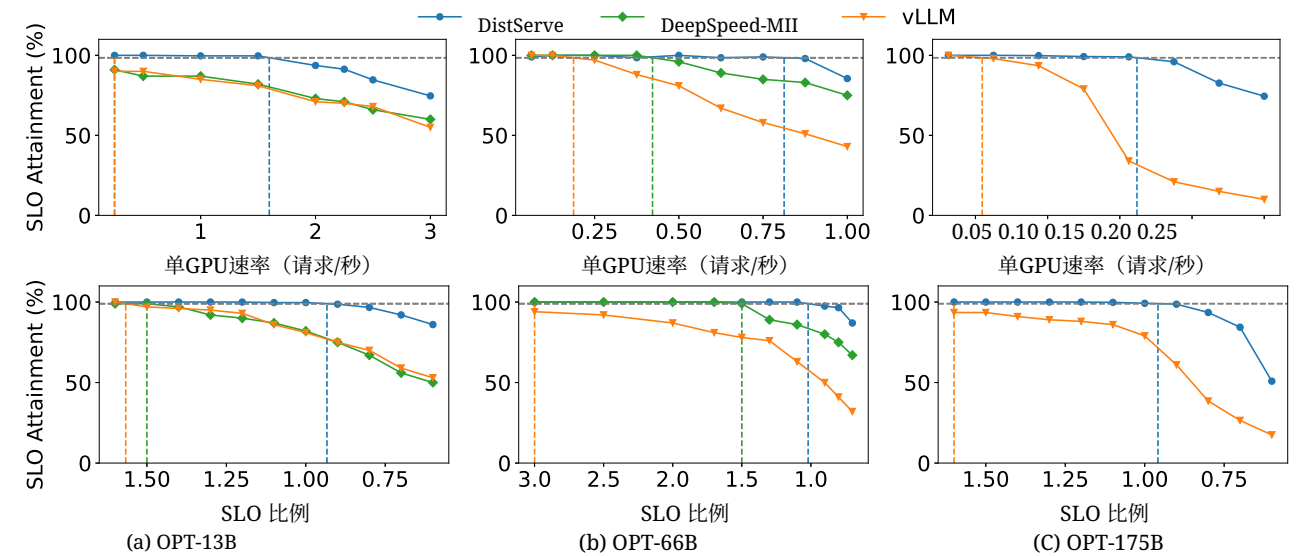


图 13: 基于OPT模型的ShareGPT数据集上的聊天机器人应用。

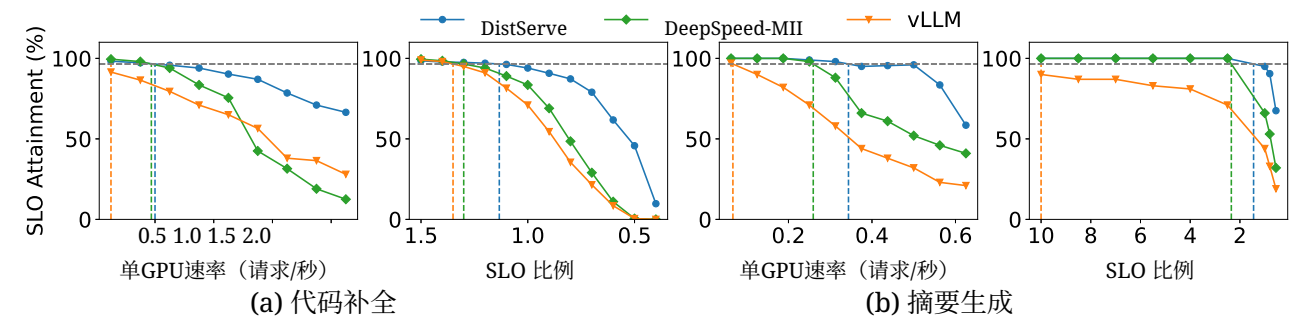


图 14: 使用OPT-66B在HumanEval和LongBench数据集上分别进行代码补全和摘要生成任务。

B 端到端实验中的DistServe部署

表3展示了在DistServe的端到端实验§6.2中为预填充和解码实例所选定的张量并行 (TP) 和流水线并行 (PP) 配置。

模型	数据集	预填充		解码	
		TP	PP	TP	PP
OPT-13B	ShareGPT	2	1	1	1
OPT-66B	ShareGPT	4	1	2	2
OPT-66B	LongBench	4	1	2	2
OPT-66B	HumanEval	4	1	2	2
OPT-175B	ShareGPT	3	3	4	3

表 3: DistServe 在端到端实验中选择的并行策略。

C 99% SLO达成率下的端到端结果

图 13 和图 14 展示了 DistServe 与基线系统在 §6.2 相同设置下的端到端性能，区别在于 SLO 达成目标调整为 99%。我们可以看到，在更严格的 SLO 达成目标下，与 vLLM 相比，DistServe 仍能维持 $3\times-8\times$ 的高速率和 $1.24\times-6.67\times$ 的更严格 SLO。与 DeepSpeed-MII 相比，DistServe 可实现 $1.32\times-8\times$ 的高速率和 $1.20\times-1.58\times$ 的更严格 SLO。